Break the Login: Atacarea și securizarea autentificării

Curs: Dezvoltarea Aplicațiilor Software Securizate
Facultatea: Facultatea de Matematică și Informatică, Universitatea din București
Student: Văsii Alexandru-Alin
Grupă: 464
Profesor: Conf. Univ. Dr. Marius Iulian Mihăilescu
Data: 21.04.2026

Cuprins:

1. Introducere 1
2. Obiective 1
3. Arhitectura aplicației Deskly/ AuthX 1
   Stack tehnologic utilizat 1
      Backend: 1
      Persistență: 1
      Biblioteci de securitate: 1
      Instrumente auxiliare: 1
   Structura bazei de date 1
      Tabelul users 1
      Tabelul tickets 1
      Tabelul audit_logs 1
   Considerente arhitecturale de securitate 1
      Schema actualizată: 1
4. Setup mediu de lucru 1
   Mediu utilizat 1
      Sistem de operare: 1
      Software principal: 1
   Instalarea componentelor necesare 1
      Rolul fiecărei componente: 1
   Structura proiectului local 1
   Rularea aplicației 1
      Varianta vulnerabilă: 1
      Varianta securizată: 1
   Testarea vulnerabilităților 1
      Brute Force: 1
      Predictable Reset Token Attack: 1
5. Implementarea versiunii vulnerabile (app_insecure.py) 1
   5.1 Vulnerabilitate: Weak Password Policy 1
      Impact: 1
      OWASP: 1

5.2 Vulnerabilitate: MD5 Password Hashing 1
Impact: 1
Cod vulnerabil: 1
5.3 Vulnerabilitate: Brute Force 1
Script PoC: 1
Impact: 1
5.4 Vulnerabilitate: User Enumeration 1
Impact: 1
5.5 Vulnerabilitate: Insecure Session Management 1
Impact: 1
Cod vulnerabil: 1
5.6 Vulnerabilitate: Predictable Password Reset Token 1
Impact: 1
Script PoC: 1
5.7 Vulnerabilitate: Broken Access Control / IDOR 1
Impact: 1
Cod vulnerabil: 1
6. Analiza vulnerabilităților și demonstrarea atacurilor (PoC) 1
Atac 1: Brute Force Login 1
Atac 2: Password Reset Token Brute Force 1
Atac 3: IDOR Ticket Editing 1
7. Implementarea versiunii securizate (app_secure.py) 1
Fix-uri implementate: 1
7.1 Password Policy 1
7.2 bcrypt 1
7.3 Generic Errors 1
7.4 Dummy Hash 1
7.5 Rate Limiting 1
7.6 Session Hardening 1
7.7 Secure Password Reset 1
7.8 Broken Access Control Fix 1
7.9 Audit Logging 1
8. Re-testarea aplicației după fix-uri 1

Brute Force: 1
Token Attack: 1
IDOR: 1
User Enumeration: 1
9. Audit logging și trasabilitate 1
Exemple: 1
Beneficii: 1
10. Tabel comparativ – Insecure vs Secure 1
11. Concluzii 1
Lecțiile pe care le-am învățat: 1
Bibliografie 1

1. Introducere

În peisajul tehnologic actual, caracterizat prin digitalizarea accelerată a proceselor de afaceri, securitatea aplicațiilor web a devenit o necesitate imperativă. Sistemele de autentificare și gestionare a accesului constituie prima linie de apărare a oricărei infrastructuri informatice, fiind totodată vectorii cei mai frecvent vizați de către atacatori.

Proiectul "Deslky - BUILD, HACK & SECURE" are scopul de a analiza în profunzime modul în care un sistem de autentificare poate fi compromis ca urmare a unor implementări deficitare, precum și demonstrarea eficacității principiilor *secure-by-design* în eliminarea acestor vulnerabilități.

În cadrul scenariului propus, am investigat securitatea unei aplicații web fictive, "AuthX", utilizată într-un context intern pentru gestionarea ticketelor de asistență tehnică. Provocarea acestui proiect constă în asumarea unui rol dublu, specific analizei de securitate moderne:

- **Rolul de Security Tester:** Implică adoptarea perspectivei unui atacator cu scopul de a identifica, analiza și exploata vulnerabilitățile critice din codul sursă. Etapa aceasta a fost necesară pentru a înțelege riscurile reale la care este expusă aplicația.
- **Rolul de Developer:** Presupune transformarea constatărilor în soluții tehnice concrete, iar obiectivul a fost rescrierea componentelor vulnerabile, implementând mecanisme de protecție moderne, în conformitate cu cele mai bune practici din prezent din industrie.

Am abordat ciclul complet de viață al unei aplicații și am examinat componentele critice de securitate, și anume: procesele de înregistrare și autentificare, gestionarea sesiunilor utilizatorilor, mecanismele de resetare a parolelor, controlul accesului pe bază de roluri (RBAC) și logarea acțiunilor pentru audit (audit logging).

Analiza mea se concentrează pe o mulțime de vulnerabilități identificate de organizația internațională OWASP (*Open Web Application Security Project*), incluzând, dar fără a se limita la:

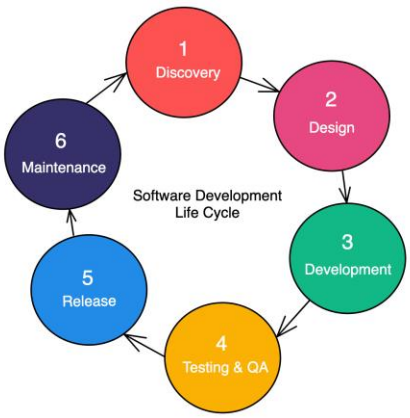
- Politici de parole deficitare (*Weak Password Policy*);
- Stocarea nesigură a credentialelor (*Insecure Password Storage*);
- Expunerea la atacuri de tip Brute Force și User Enumeration;
- Configurări incorecte ale sesiunilor (*Session Misconfiguration*);
- Vulnerabilități în procesele de recuperare a contului (*Insecure Password Reset*);
- Deficiențe în controlul accesului, în special referințe nesigure la obiecte (*Broken Access Control – IDOR*).

Voi justifica pe parcursul acestui raport, în paralel cu documentarea procesului de "fix", fiecare decizie de securitate luată va fi pentru a putea înțelege importanța unei abordări de tip *defense-in-depth* pentru protejarea integrității, confidențialității și disponibilității datelor.

2. Obiective

Scopul acestui proiect a fost de a simula un ciclu complet de dezvoltare software securizată (Secure SDLC), trecând prin fazele de proiectare, analiză de vulnerabilități, exploatare și remediere. Proiectul a fost structurat pe următoarele obiective strategice:

- **Arhitecturarea și Construirea Aplicației AuthX:** Dezvoltarea unui sistem funcțional de autentificare și gestionare a ticketelor (ticketing system), capabil să susțină fluxurile de lucru de bază (Înregistrare, Login, Logare acțiuni).
- **Modelarea Vulnerabilităților (Insecure Design):** Implementarea intenționată a unei versiuni vulnerabile, configurată pentru a expune punctele critice de securitate în fața atacurilor externe. Această etapă a funcționat ca un mediu controlat de testare (sandbox), destinat reproducerii unor scenarii realiste de compromitere.
- **Evaluarea Vulnerabilităților:** Identificarea și exploatarea practică a lacunelor de securitate (Brute Force, User Enumeration, Predictable Password Reset Tokens, Broken Access Control / IDOR și Session Misconfiguration) prin utilizarea unor scripturi de tip Proof-of-Concept (PoC). Obiectivul a fost demonstrarea impactului concret pe care un atacator l-ar putea avea asupra integrității datelor.
- **Remedierea și Securizarea:** Aplicarea controalelor de securitate de tip *industry-standard* pentru a neutraliza vectorii de atac identificați anterior, transformând aplicația dintr-o entitate vulnerabilă într-una robustă.
- **Validarea Contramăsurilor:** Confirmarea faptului că scripturile PoC utilizate în faza de atac nu mai produc rezultate pozitive în versiunea securizată, validând astfel eficacitatea fix-urilor.
- **Trasabilitatea prin Audit:** Implementarea și demonstrarea funcționalității de *Audit Logging*, esențială pentru monitorizarea activității utilizatorilor și pentru investigațiile de tip *forensics* în cazul unor incidente de securitate.



3. Arhitectura aplicației Deskly/ AuthX

Aplicația AuthX a fost proiectată și implementată utilizând o arhitectură web modulară, construită pentru a simula atât funcționalitatea unui sistem real de autentificare enterprise, cât și vulnerabilitățile frecvent întâlnite în practică. Alegerea tehnologiilor a urmărit un echilibru între simplitatea implementării, realismul procesului de securizare și posibilitatea reproducerii locale într-un mediu controlat.

Backend-ul aplicației a fost dezvoltat în limbajul Python, utilizând framework-ul Flask, ales datorită flexibilității sale, ușurinței de configurare și popularității în dezvoltarea aplicațiilor web de dimensiuni mici și medii. Flask a permis implementarea rapidă a fluxurilor critice de autentificare, gestionare sesiuni și control al accesului, oferind în același timp control direct asupra logicii de securitate.

Pentru persistența datelor a fost utilizat SQLite3, o bază de date relațională lightweight, potrivită pentru dezvoltare locală și testare academică. SQLite a fost ales datorită portabilității, simplității administrative și posibilității de a demonstra realist fluxuri de autentificare, autorizare și audit fără necesitatea configurării unui server dedicat.

În ceea ce privește securitatea credentialelor, proiectul a inclus două abordări distincte:

- **bcrypt** – utilizat în varianta securizată (app_secure.py), pentru hashing adaptiv al parolelor, incluzând salt automat și rezistență sporită la atacuri de tip brute force;
- **hashlib (MD5)** – utilizat exclusiv în varianta vulnerabilă (app_insecure.py), cu scop didactic, pentru a evidenția slăbiciunile algoritmilor criptografici depășiți în protejarea parolelor.

Stack tehnologic utilizat

Backend:

- Python 3.x
- Flask

Persistență:

- SQLite3

Biblioteci de securitate:

- bcrypt
- hashlib (MD5 – exclusiv pentru demonstrarea vulnerabilităților)

Instrumente auxiliare:

- Git / GitHub (versionare și trasabilitate cod sursă)
- Curl / Browser web (testare endpoint-uri și request/response analysis)
- Requests (automatizare PoC pentru brute force și token attacks)

Fig. 1: Structura proiect pe GitHub

Aplicația implementează un flux complet de Identity & Access Management (IAM), incluzând:

- Register
- Login
- Logout
- Forgot Password
- Reset Password
- Dashboard
- Ticket System

Structura bazei de date

Modelul de date a fost conceput pentru a susține atât funcționalitățile de business, cât și cerințele de securitate.

Tabelul `users`

Conține identitatea utilizatorilor și componentele de securitate asociate:

- `id` – identificator unic;
- `email` – username unic;
- `password_hash` – hash-ul parolei;
- `role` – implementarea RBAC (ANALYST / MANAGER);
- `created_at` – trasabilitate temporală;
- `locked` – suport pentru blocarea contului;
- `reset_token` – mecanism resetare parolă;
- `token_expiry` – expirarea token-ului.

Tabelul `tickets`

Susține sistemul de ticketing și permite demonstrarea controlului accesului:

- `id`
- `title`
- `description`
- `severity`
- `status`
- `owner_id` – coloană critică pentru prevenirea IDOR / Broken Access Control.

Tabelul `audit_logs`

Asigură monitorizarea și forensic readiness:

- `id`
- `user_id`
- `action`
- `resource`
- `timestamp`
- `ip_address`

Considerente arhitecturale de securitate

- Relația `users → tickets` permite verificarea ownership-ului și aplicarea autorizării server-side, lucru esențial pentru prevenirea accesului neautorizat la resurse.
- Relația `users → audit_logs` oferă accountability și trasabilitate completă asupra acțiunilor, ceea ce permite detectarea incidentelor și investigațiile de tip forensic.

- Prin separarea explicită a componentelor de autentificare, autorizare, recuperare cont și audit, aplicația a fost construită astfel încât să permită atât modelarea vulnerabilităților, cât și remedierea lor într-un mod structurat, conform principiilor Secure Software Development Lifecycle (Secure SDLC).

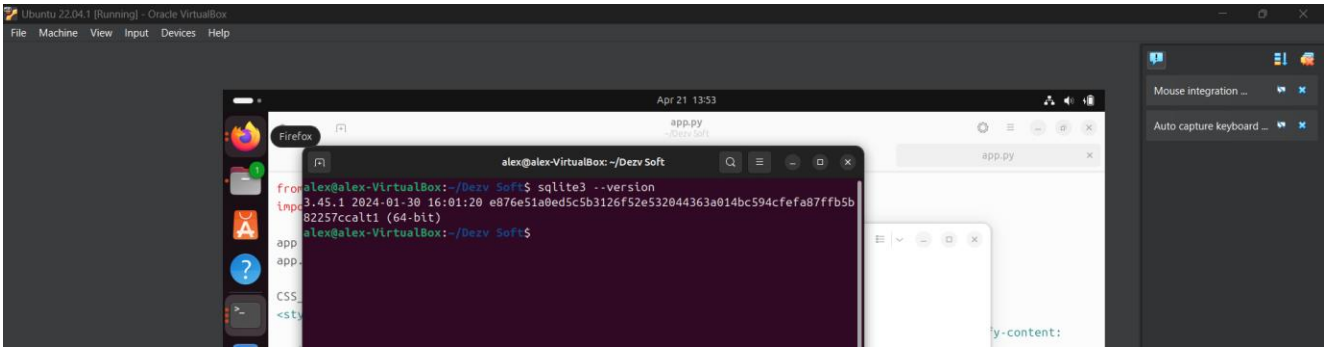


Fig. 2: Versiunea de SQLite folosită

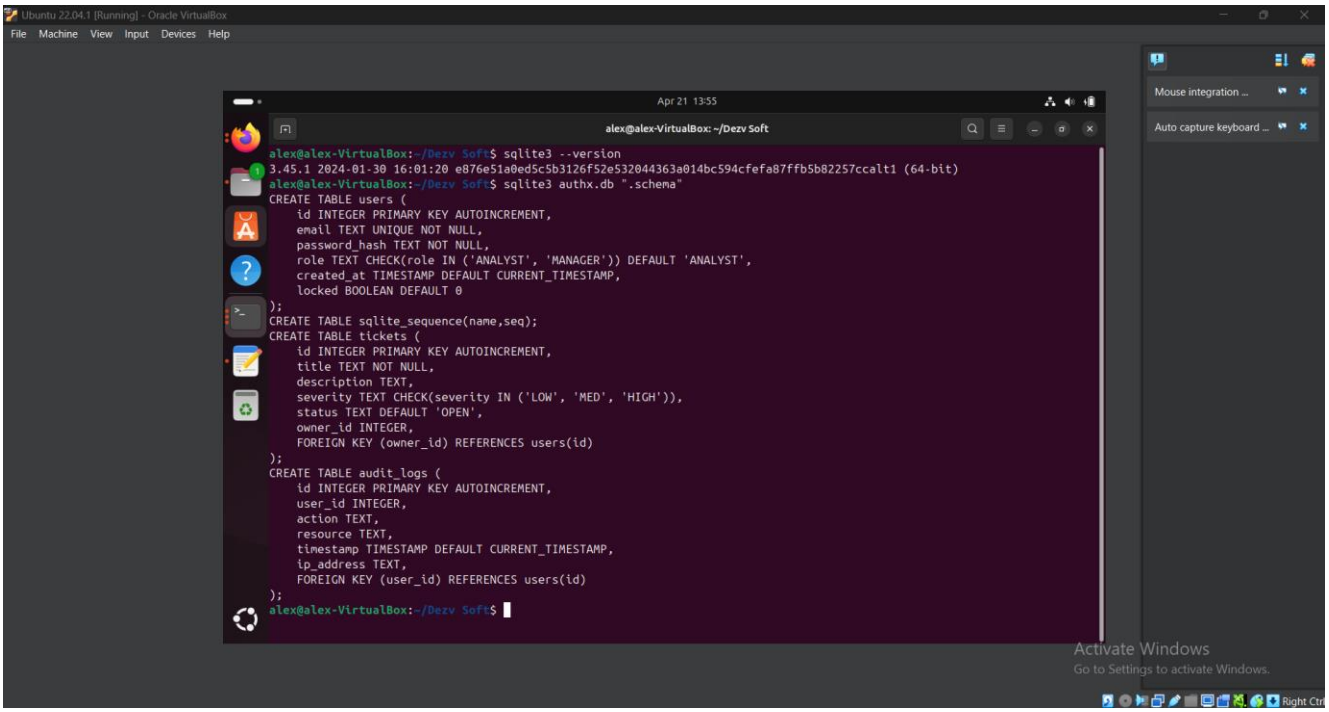


Fig. 3: Schema inițială a bazei de date pentru deskly/ AuthX

Schema actualizată:

```
CREATE TABLE users (
   id INTEGER PRIMARY KEY AUTOINCREMENT,
   email TEXT UNIQUE NOT NULL,
   password_hash TEXT NOT NULL,
   role TEXT CHECK(role IN ('ANALYST', 'MANAGER')) DEFAULT 'ANALYST',
   created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
   locked BOOLEAN DEFAULT 0
, reset_token TEXT, token_expiry DATETIME);
CREATE TABLE sqlite_sequence(name,seq);
CREATE TABLE tickets (
   id INTEGER PRIMARY KEY AUTOINCREMENT,
   title TEXT NOT NULL,
   description TEXT,
   severity TEXT CHECK(severity IN ('LOW', 'MED', 'HIGH')),
   status TEXT DEFAULT 'OPEN',
   owner_id INTEGER,
   FOREIGN KEY (owner_id) REFERENCES users(id)
);
CREATE TABLE audit_logs (
   id INTEGER PRIMARY KEY AUTOINCREMENT,
   user_id INTEGER,
   action TEXT,
   resource TEXT,
   timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
   ip_address TEXT,
   FOREIGN KEY (user_id) REFERENCES users(id));
```

4. Setup mediu de lucru

Pentru implementarea, testarea și demonstrarea proiectului a fost utilizat un mediu virtualizat controlat, cu scopul de a asigura reproductibilitate, trasabilitate și conformitate cu cerințele baremului de evaluare. Utilizarea unei mașini virtuale (VM) a permis izolarea mediului de testare și simularea realistă a unui workflow profesional de dezvoltare și security assessment.

Mediu utilizat

Sistem de operare:

- Ubuntu Linux (instalat pe Oracle VirtualBox)

Software principal:

- Python 3.x
- Flask
- SQLite3
- bcrypt
- Git
- Browser web - FireFox
- curl

Mediul virtualizat oferă multiple avantaje în contextul unui proiect de securitate:

- separarea completă de sistemul gazdă;
- posibilitatea capturilor autentice cu username și hostname;
- testare sigură a atacurilor PoC;
- reproductibilitate pentru demonstrații;
- respectarea cerințelor de audit academic.

Instalarea componentelor necesare

După configurarea mediului Ubuntu, am instalat dependențele necesare aplicației:

```
sudo apt update
sudo apt install python3 python3-pip sqlite3 python3-flask python3-
bcrypt -y
pip install requests
```

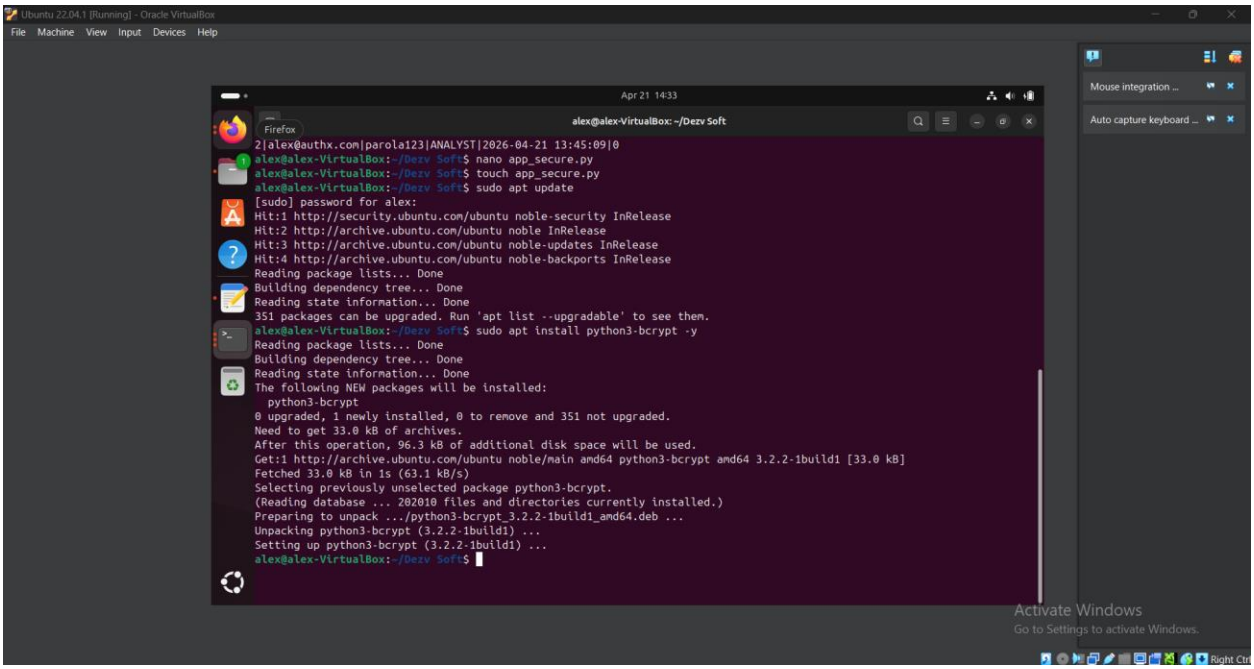


Fig. 4: Instalarea pachetelor

Rolul fiecărei componente:

- `Flask` – framework backend
- `bcrypt` – hashing parole secure
- `requests` – automatizare scripturi PoC
- `sqlite3` – administrare și verificare bază de date
- `git` – versionare și livrare proiect

Structura proiectului local

Directorul proiectului conține următoarele fișiere principale:

```
app_insecure.py
app_secure.py
brute_force.py
brute_token.py
authxinsecure.db
authx.db
```

Această structură permite separarea clară între:

- versiunea vulnerabilă
- versiunea securizată
- instrumentele de atac
- bazele de date dedicate fiecărei versiuni

Rularea aplicației

Varianta vulnerabilă:

```
python3 app_insecure.py
```

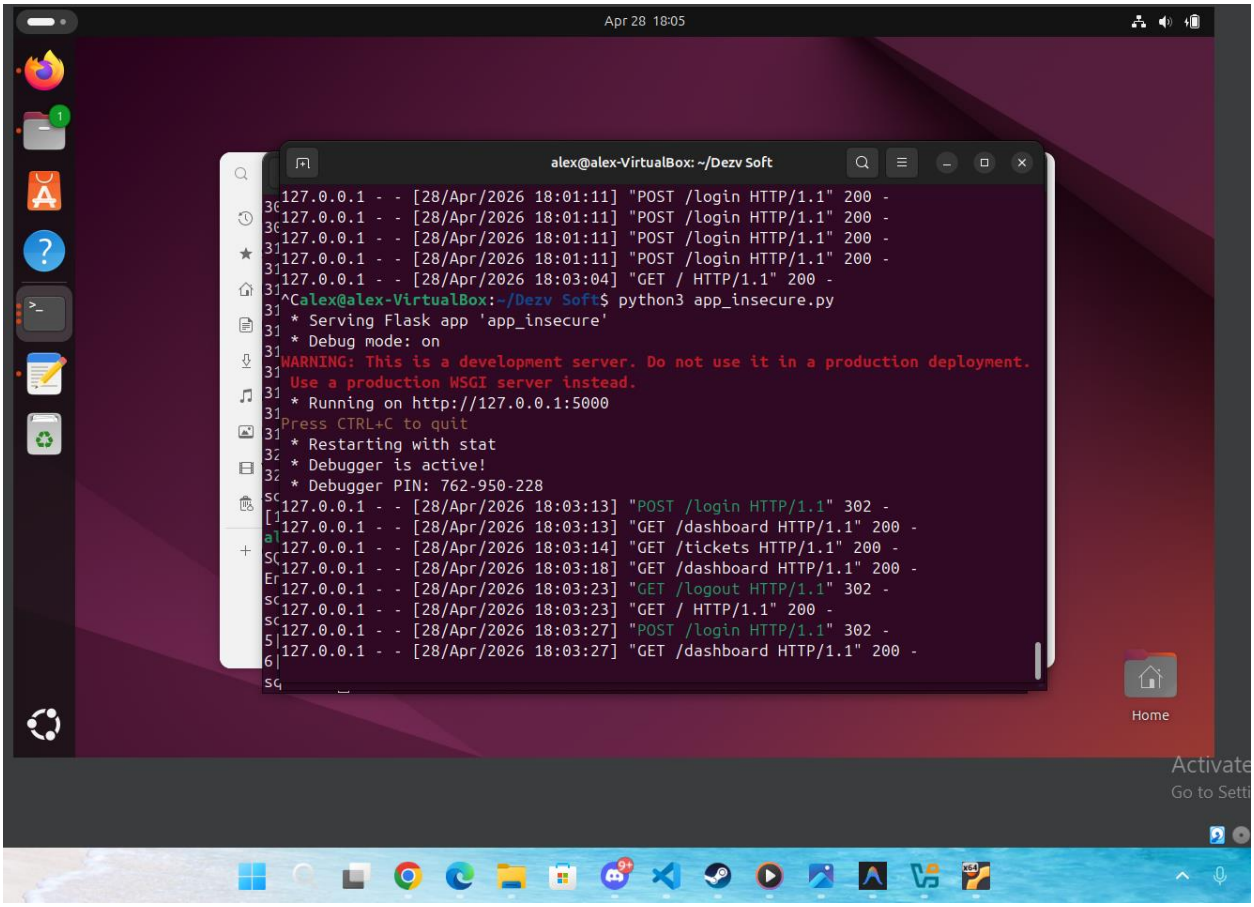


Fig. 5: Rulez app_insecure.py

Varianta securizată:

```
python3 app_secure.py
```

Aplicația rulează local pe:

http://localhost:5000

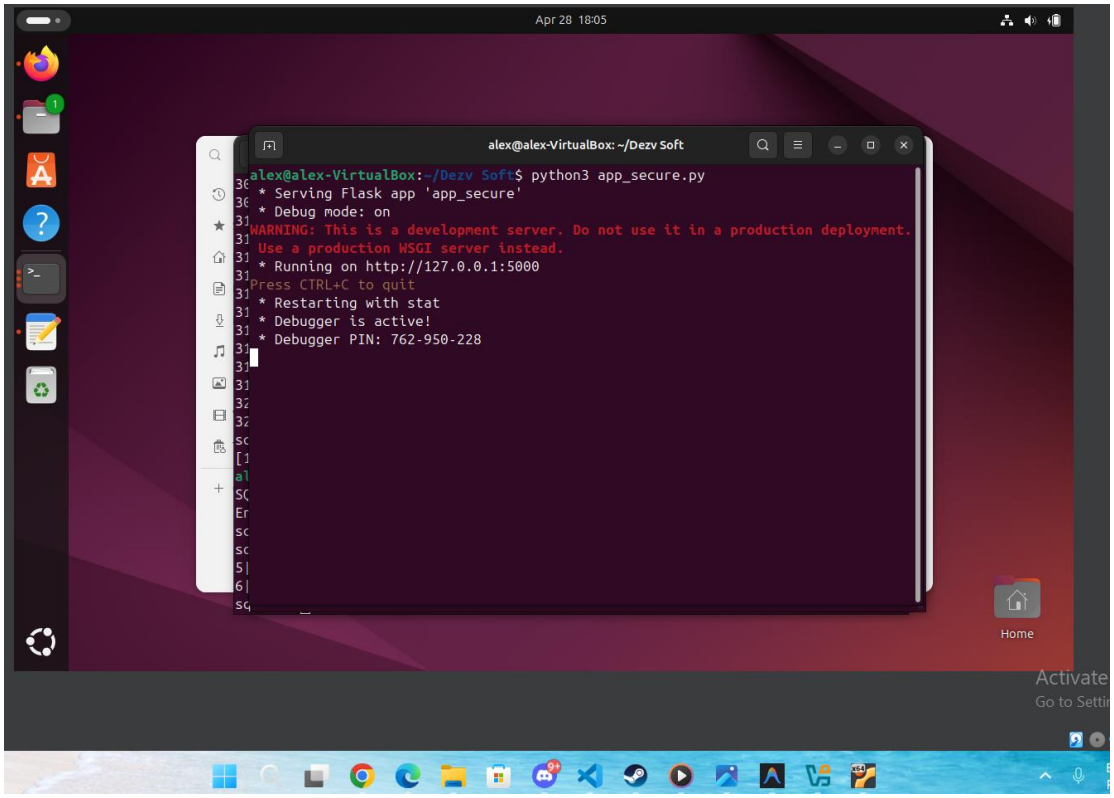


Fig. 6: Rulez app_secure.py

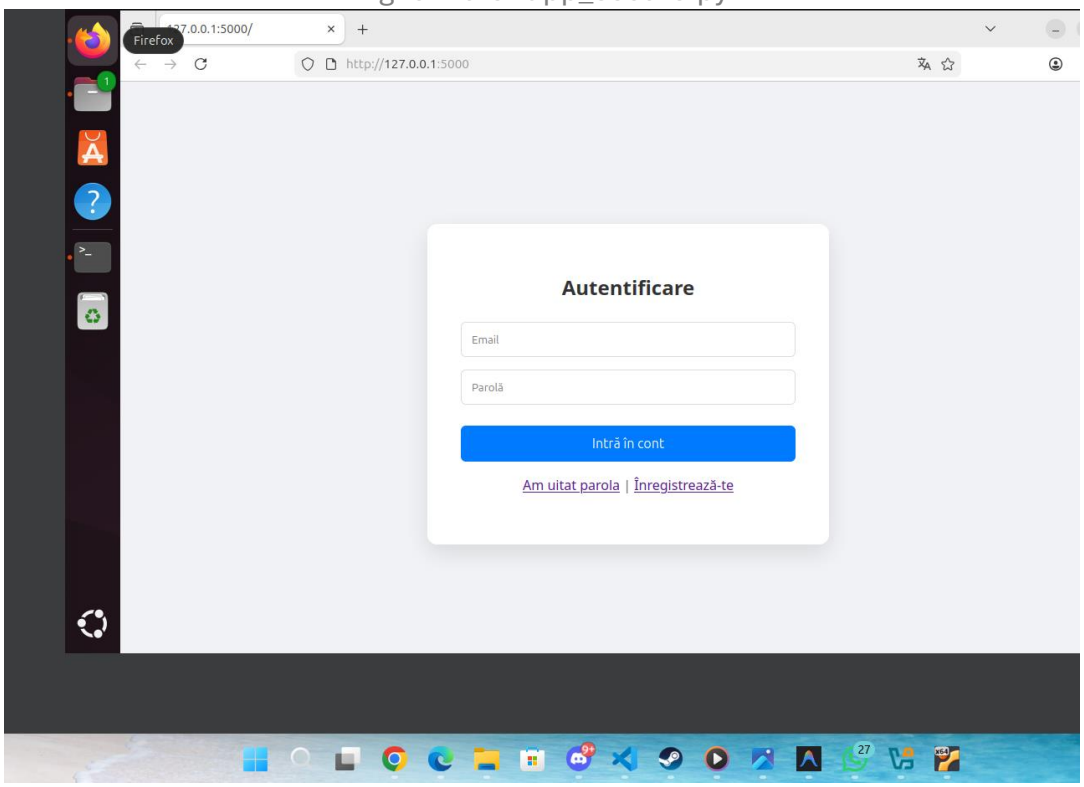


Fig. 7: Pagina de start arată la fel pentru ambele versiuni ale aplicației

Testarea vulnerabilităților

Brute Force:

```
python3 brute_force.py
```

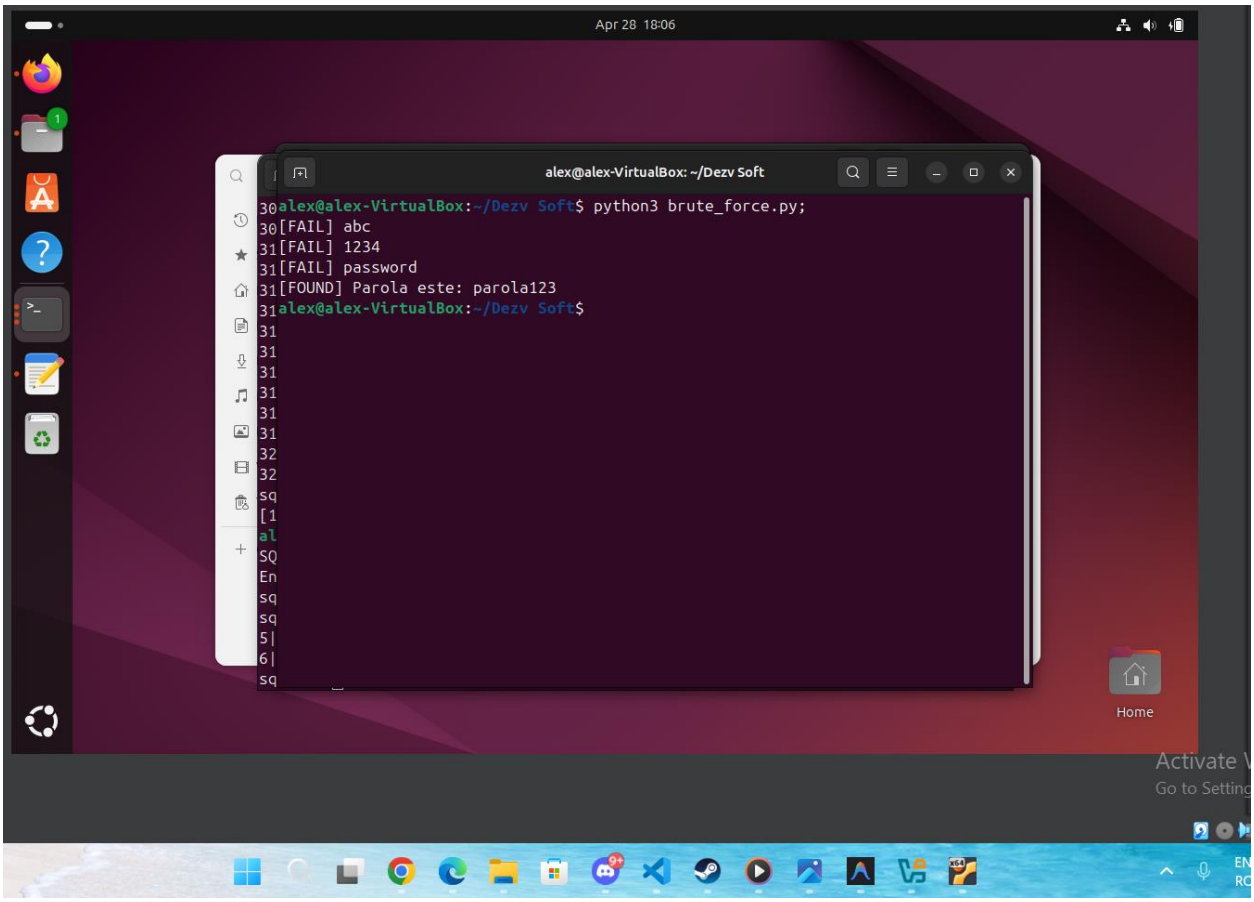


Fig. 8: Atacul de brute_force se desfășoară cu succes pe versiunea vulnerabilă

Am realizat acest script pentru a demonstra riscul expunerii la atacuri de tip *Credential Stuffing* sau *Dictionary Attack*.

Logica din spatele atacului: Vulnerabilitatea există în absența oricărui mecanism de "Rate Limiting" (limitarea numărului de cereri) sau de "Account Lockout" (blocarea contului după X încercări eșuate). Într-o implementare securizată, sistemul ar trebui să detecteze un volum neobișnuit de cereri de autentificare de la același IP sau pentru același utilizator și să aplice măsuri de atenuare (ex: CAPTCHA, delay, blocare temporară).

Funcționarea scriptului:

1. **Iterație:** Scriptul utilizează o listă predefinită de parole (dicționar).
2. **Cerere HTTP:** Trimite cereri POST către endpoint-ul `/login` al aplicației.

Analiza răspunsului: Scriptul monitorizează codul de status HTTP sau mesajul returnat. Dacă aplicația returnează un cod de succes (ex: 302 Redirect sau 200 OK cu un cookie de sesiune), atacul este considerat reușit.

Predictable Reset Token Attack:

```
python3 brute_token.py
```

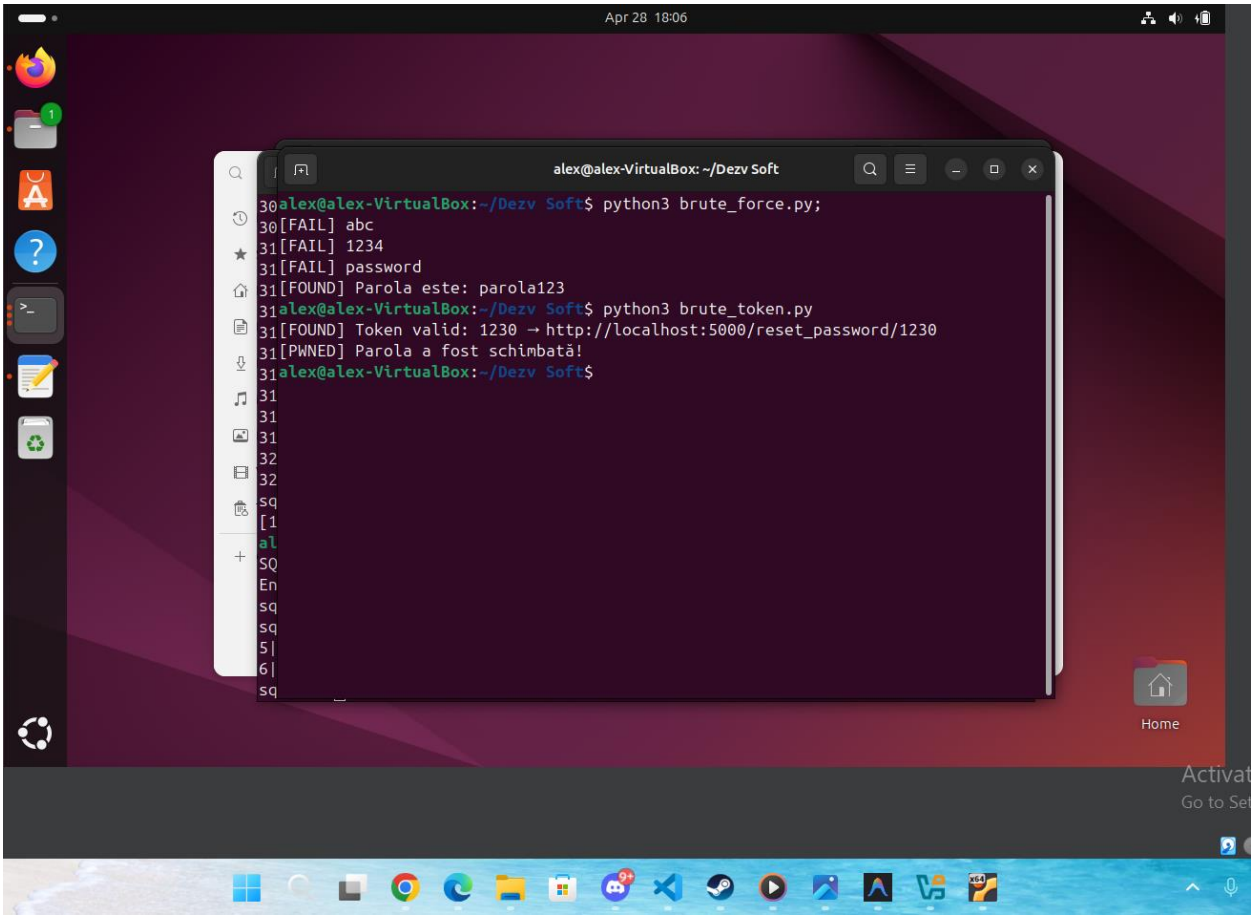


Fig. 9: Atacul de brute_token se desfășoară cu succes pe versiunea vulnerabilă

Acest script vizează o vulnerabilitate mai subtilă, dar relevantă, și anume, predictibilitatea token-urilor de resetare a parolei.

Logica din spatele atacului: În fluxul de "Forgot Password", aplicația generează un token temporar trimis utilizatorului pentru a valida identitatea. Dacă acest token este scurt (ex: 4-6 cifre) sau generat folosind o bibliotecă de numere pseudo-aleatorii nesigură (de tip `random` în loc de `secrets`), un atacator poate încerca toate combinațiile posibile până o găsește pe cea corectă. Aceasta este o formă de *Insecure Direct Object Reference (IDOR)* combinată cu *Insufficient Entropy*.

Funcționarea scriptului:

3. **Targetare:** Scriptul vizează utilizatorul țintă care a solicitat resetarea.
4. **Generare Payload:** Se generează secvențial sau aleatoriu toate valorile posibile pentru token-ul de resetare.
5. **Exploatare:** Trimite cereri către endpoint-ul de validare (`/reset_password/<token>`).
6. **Verificare:** Identifică validarea cu succes a token-ului, permițând atacatorului să schimbe parola utilizatorului fără a avea acces la email-ul acestuia.

Aceste scripturi au fost utilizate pentru a demonstra reproducibil vulnerabilitățile și pentru a valida ulterior eficacitatea măsurilor defensive implementate.

5. Implementarea versiunii vulnerabile (app_insecure.py)

Versiunea inițială (`app_insecure.py`) a fost construită intenționat cu multiple vulnerabilități pentru a demonstra realismul atacurilor, un studiu de caz pentru "așa nu".

5.1 Vulnerabilitate: Weak Password Policy

În versiunea vulnerabilă, procesul de înregistrare a utilizatorilor (`/register`) nu impune nicio restricție asupra complexității parolei. Aplicația acceptă orice șir de caractere, indiferent de lungime sau compoziție.

Impact:

Lipsa validării permite utilizatorilor să aleagă parole triviale, precum:

- 1234
- password
- Test

Aceste parole sunt primele testate de scripturile automate în atacurile de tip "Credential Stuffing", iar un atacator poate compromite un procent mare de conturi într-un timp scurt folosind dicționare de parole comune.

OWASP:

A07: Identification and Authentication Failures

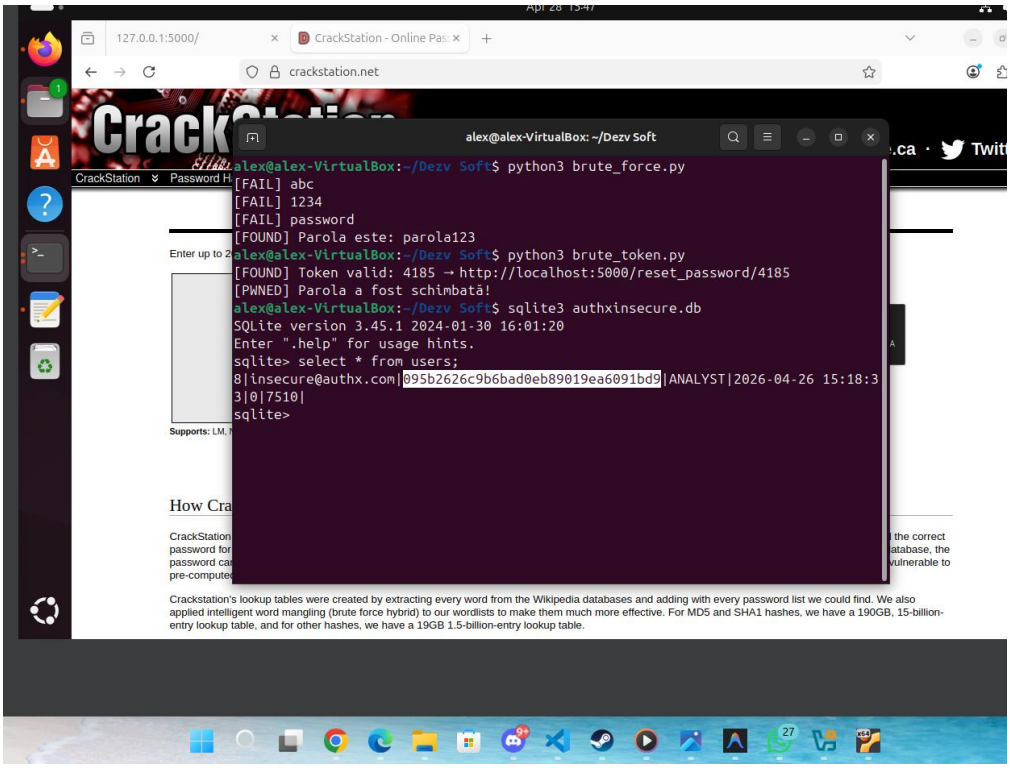


Fig. 10: Parola generică, slab hash-uita (MD5)

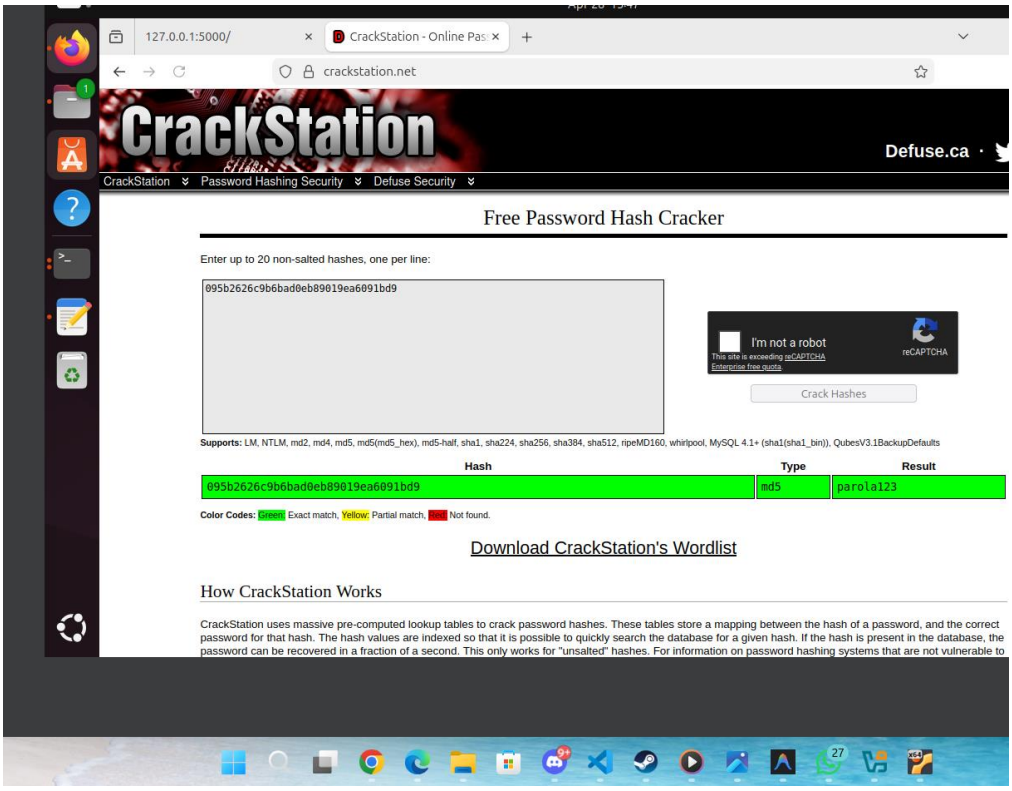


Fig. 11: Spargerea parolei slab hash-uite (MD5)

5.2 Vulnerabilitate: MD5 Password Hashing

Aplicația utilizează algoritmul MD5 pentru a genera hash-urile parolelor înainte de stocarea în baza de date `authxinsecure.db`.

MD5 este considerat un algoritm extrem de rapid și ușor de spart, nu utilizează un mecanism de "salting" (adăugarea unui șir aleatoriu unic pentru fiecare utilizator), iar două parole identice vor genera același hash.

Impact:

- Rainbow tables
- Hash cracking rapid
- Lipsa salt

Cod vulnerabil:

```
hashed = hashlib.md5(new_password.encode('utf-8')).hexdigest()
```

5.3 Vulnerabilitate: Brute Force

Endpoint-ul de login nu implementează mecanisme de tip *Rate Limiting* sau *Account Lockout*. Un utilizator (sau un script) poate trimite un număr infinit de cereri de autentificare fără a fi blocat de server.

Script PoC:

`brute_force.py`

Impact:

Se pot rula sute de combinații utilizator/parolă pe minut până la identificarea credențialelor corecte, iar serverul răspunde cu aceeași viteză la fiecare cerere, facilitând atacul.

5.4 Vulnerabilitate: User Enumeration

Dacă un atacator introduce un email care nu există, aplicația răspunde cu "User inexistent". Dacă email-ul există, dar parola este greșită, răspunde cu "Parolă incorectă".

Impact:

Aceste mesaje distincte confirmă atacatorului dacă un anumit email este înregistrat în sistem. Odată confirmat email-ul, atacatorul își poate concentra toate resursele de *brute force* doar pe acel cont, crescând rata de succes.

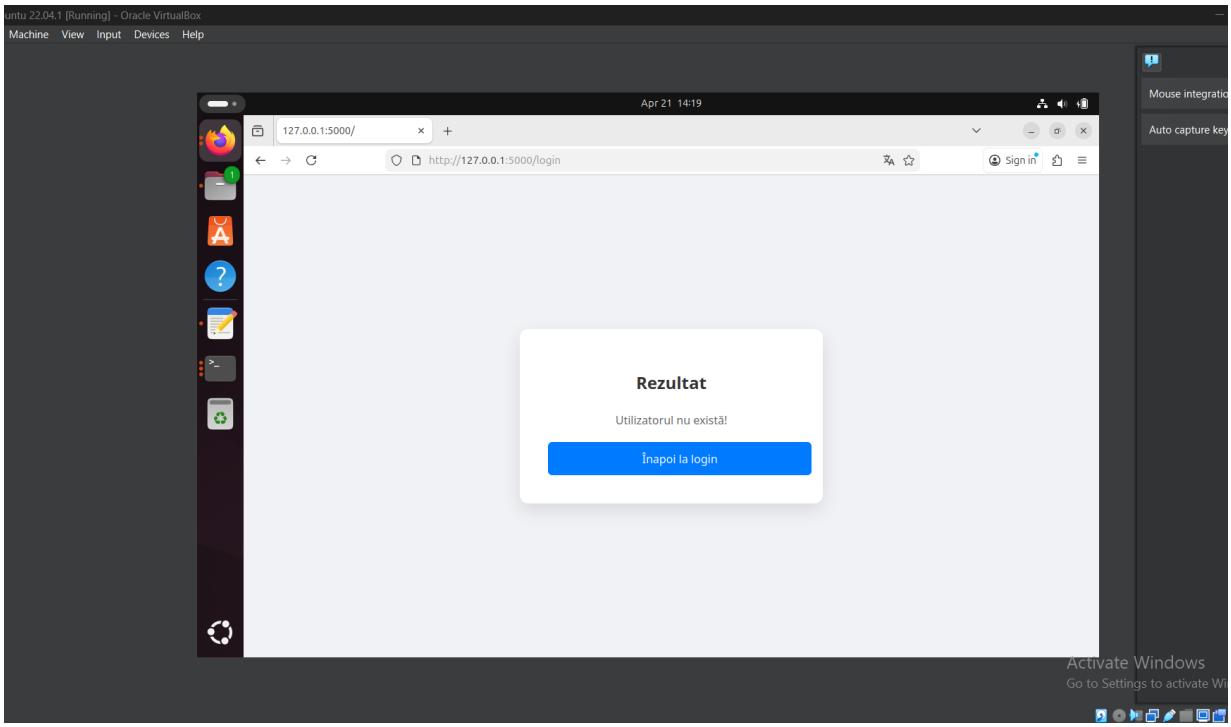


Fig. 12: Mesaj specific pentru user inexistent

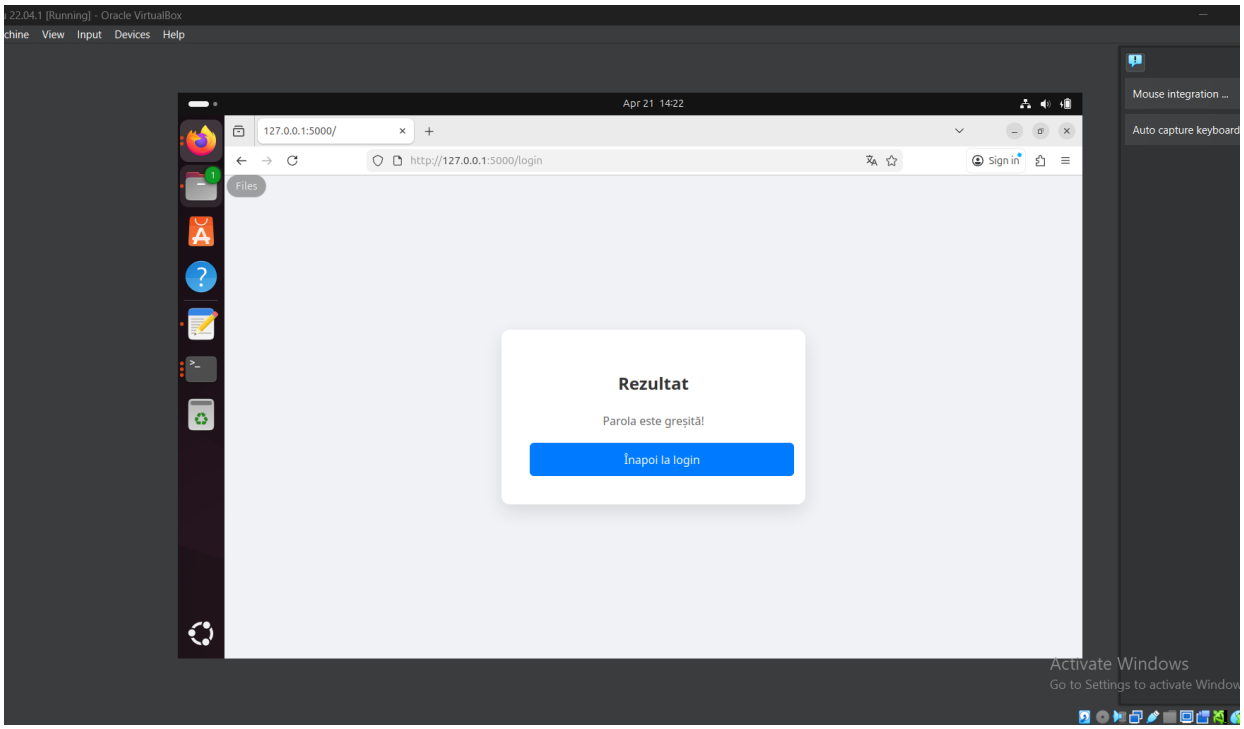


Fig. 13: Mesaj specific pentru parola greșită

5.5 Vulnerabilitate: Insecure Session Management

Configurarea obiectului `session` în Flask este lăsată pe valorile implicite sau configurată nesecurizat:

- `SESSION_COOKIE_HTTPONLY=False`: Permite scripturilor JavaScript să acceseze cookie-ul de sesiune.
- `SESSION_COOKIE_SAMESITE='Lax'`: Expunere la atacuri de tip CSRF.
- Utilizarea unei chei `secret_key` previzibile sau hardcodate.

Impact:

- Session hijacking
- Cookie theft
- XSS risk

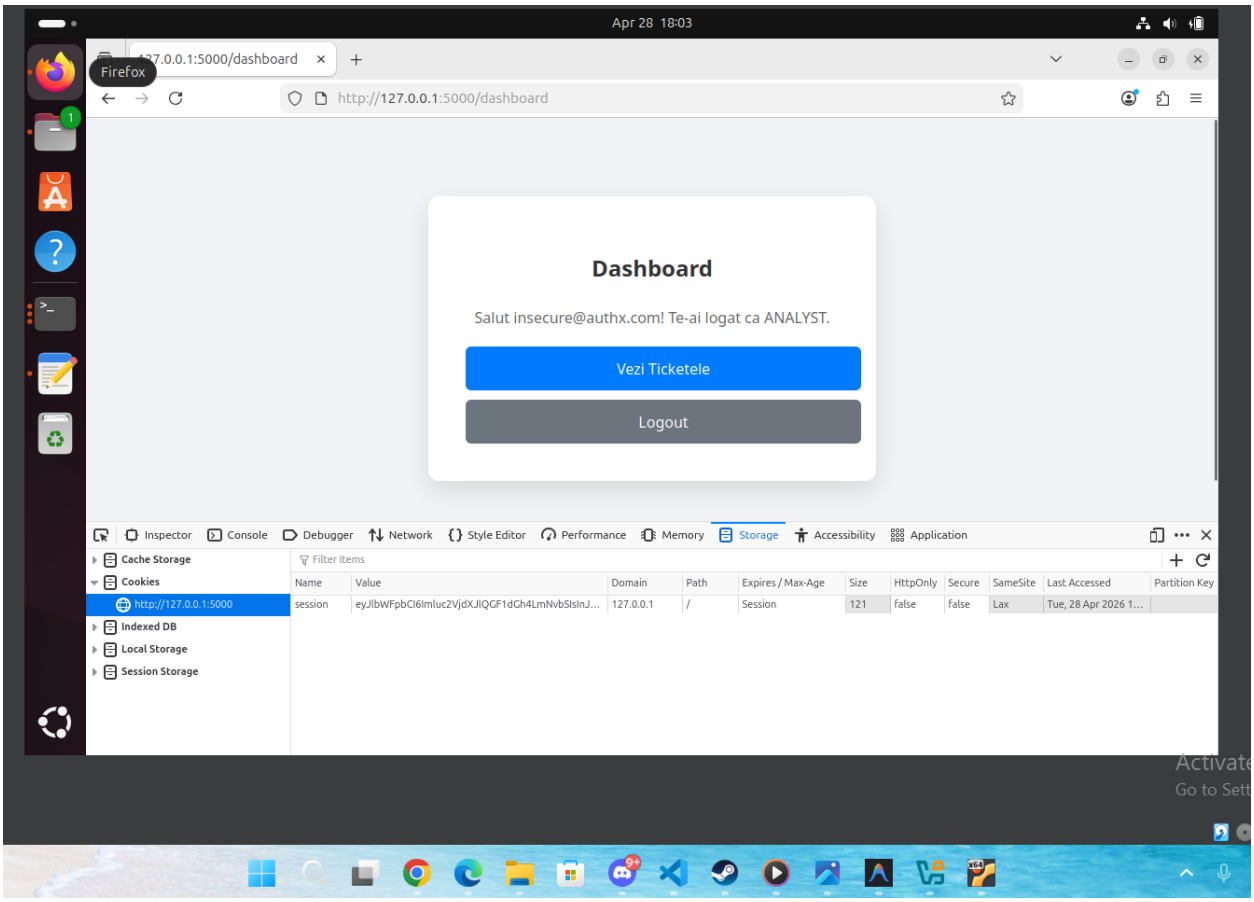


Fig. 14: Configurare sesiune insecure

Cod vulnerabil:

```
app.config.update(
    SESSION_COOKIE_HTTPONLY=False,
    SESSION_COOKIE_SECURE=False,
```

```
    SESSION_COOKIE_SAMESITE='Lax',
)
```

5.6 Vulnerabilitate: Predictable Password Reset Token

Mecanismul de resetare a parolei generează un token numeric de doar 4 cifre folosind biblioteca `random` din Python, care nu este sigură din punct de vedere criptografic pentru astfel de scopuri.

```
        if user:
            token = str(random.randint(1000, 9999))
```

Impact:

Deoarece există doar 9000 de combinații posibile, scriptul `brute_token.py` poate ghici token-ul în mai puțin de un minut, iar un atacator poate solicita resetarea parolei pentru orice cont și apoi se poate folosi de acel token pentru a seta o parolă nouă, eliminând necesitatea accesului la email-ul victimei.

Script PoC:

`brute_token.py`

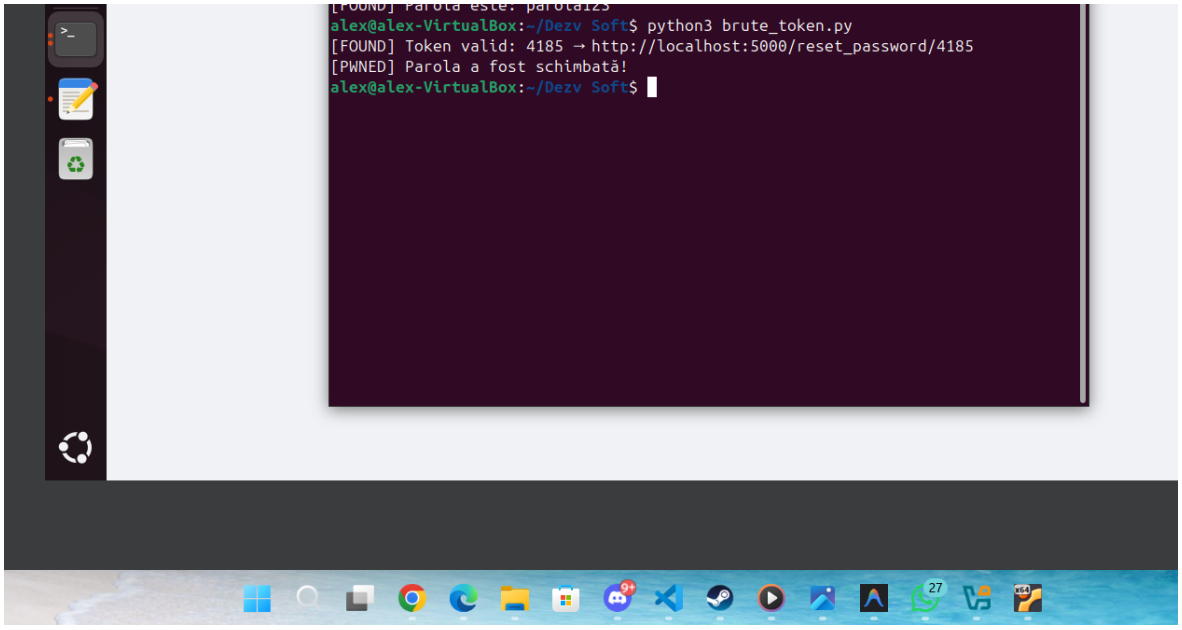


Fig. 15: Atacul brute_token are succes pe versiunea vulnerabilă

5.7 Vulnerabilitate: Broken Access Control / IDOR

Aplicația suferă de *Insecure Direct Object Reference*. Deși un utilizator are rolul de ANALYST, acesta poate accesa ticketele altor colegi sau chiar ale managerilor prin simpla modificare a ID-ului în URL (ex: `/edit_ticket/5`). Serverul preia ID-ul din URL și interoghează baza de date fără a verifica dacă owner_id al acelui ticket corespunde cu id-ul utilizatorului logat în sesiune.

Impact:

Scurgerea de informații confidențiale și posibilitatea de a modifica datele altor utilizatori (Escaladare de privilegii orizontală).

Cod vulnerabil:

```
# Interogarea extrage ticketul doar pe baza ID-ului din URL
        db.execute("UPDATE tickets SET title = ?, description = ?, status = ?,
severity = ? WHERE id = ?",
                   (title, description, status, severity, ticket_id))
```

6. Analiza vulnerabilităților și demonstrarea atacurilor (PoC)

Această etapă constă în validarea empirică a defectelor de securitate prin utilizarea unor scripturi de tip Proof of Concept (PoC). Obiectivul este de a obține acces, și voi documenta procesul prin care o configurare defectuoasă permite compromiterea triadei CIA (Confidențialitate, Integritate, Disponibilitate) ;).

Atac 1: Brute Force Login

Atacul de tip Brute Force reprezintă încercarea sistematică de a ghici credențialele unui utilizator prin testarea succesivă a unor combinații de parole. În aplicația `app_insecure.py`, succesul acestui atac este garantat de absența mecanismelor de "Rate Limiting" și de politica de parole excesiv de permisivă.

Pentru această demonstrație, am utilizat scriptul personalizat `brute_force.py`. Metodologia a urmat pașii de mai jos:

1. **Pregătirea Dicționarului:** Am creat o listă de parole comune (top 100 parole frecvente) care include termeni precum `123456`, `password`, `admin123`.

```
PASSWORDS = ["abc", "1234", "password", "parola123", "test123"]
```

2. **Automatizarea Cererilor:** Scriptul utilizează biblioteca Python `requests` pentru a trimite cereri repetitive de tip POST către endpoint-ul `/login`.

```
TARGET = "http://localhost:5000/login"
# câteva linii mai târziu ....
r = requests.post(TARGET, data={"email": EMAIL, "password": pwd})
```

3. **Monitorizarea Răspunsului:** Deoarece serverul nu blochează adresa IP după tentative eșuate, scriptul poate rula cu o viteză mare. Succesul este identificat prin analiza codului de stare HTTP (redirect către dashboard) sau prin prezența cookie-ului de sesiune în răspuns.

```
if "Dashboard" in r.text or "redirect" in r.url:
        print(f"[FOUND] Parola este: {pwd}")
        break
```

Atacul s-a încheiat cu succes în mai puțin de 30 de secunde, reușind să identifice parola contului țintă. Am demonstrat că un utilizator cu intenții malicioase poate compromite orice cont care nu folosește o parolă complexă, din cauza lipsei protecției la nivel de server.

Atac 2: Password Reset Token Brute Force

Mecanismul de resetare a parolei este una dintre cele mai critice funcționalități, iar în varianta vulnerabilă, am utilizat un token format din doar 4 cifre (1000–9999). Din punct de vedere matematic, acest spațiu de căutare este mult prea mic pentru a oferi securitate împotriva automatizării.

Prin scriptul `brute_token.py`, am simulat un atacator care cunoaște email-ul victimei:

1. **Declanșarea Resetării:** Se trimite o cerere legitimă de resetare a parolei pentru user-ul țintă, forțând serverul să genereze un token în baza de date.

```
for token in range(1000, 10000):
    url = f"http://localhost:5000/reset_password/{token}"
    r = requests.get(url)
```

2. **Iterarea Automată:** Scriptul parcurge liniar intervalul de la 1000 la 9999, trimițând cereri către `/reset_password/<token>`.
3. **Identificarea Validității:** Când scriptul întâlnește token-ul stocat în `authxinsecure.db`, serverul acceptă cererea și permite schimbarea parolei.

```
if "Token invalid" not in r.text:
        print(f"[FOUND] Token valid: {token} → {url}")

        r2 = requests.post(url, data={"password": "Hacked123"})
```

```
        print("[PWNED] Parola a fost schimbată!")
```

Resetarea neautorizată a fost realizată cu succes. Atacatorul a preluat controlul total al contului fără a avea acces la căsuța de e-mail a victimei, invalidând întregul scop al verificării prin token.

Atac 3: IDOR Ticket Editing

Vulnerabilitatea IDOR apare atunci când o aplicație folosește un identificator furnizat de utilizator pentru a accesa direct un obiect (în acest caz, un ticket de suport), fără a verifica dacă utilizatorul are permisiunea de a interacționa cu acea resursă.

Spre deosebire de atacurile anterioare care au folosit scripturi, acesta a fost realizat prin manipularea directă a cererilor HTTP:

1. **Identificarea Structurii URL:** Am observat că ticketele sunt editabile la adresa `/edit_ticket/<id>`.
2. **Manipularea Parametrului:** Fiind logat cu un cont de tip ANALYST (`test@test.com`) care nu deține tickete, am modificat manual URL-ul în browser către `/edit_ticket/8` și am putut edita ticketele atribuite altui utilizator.

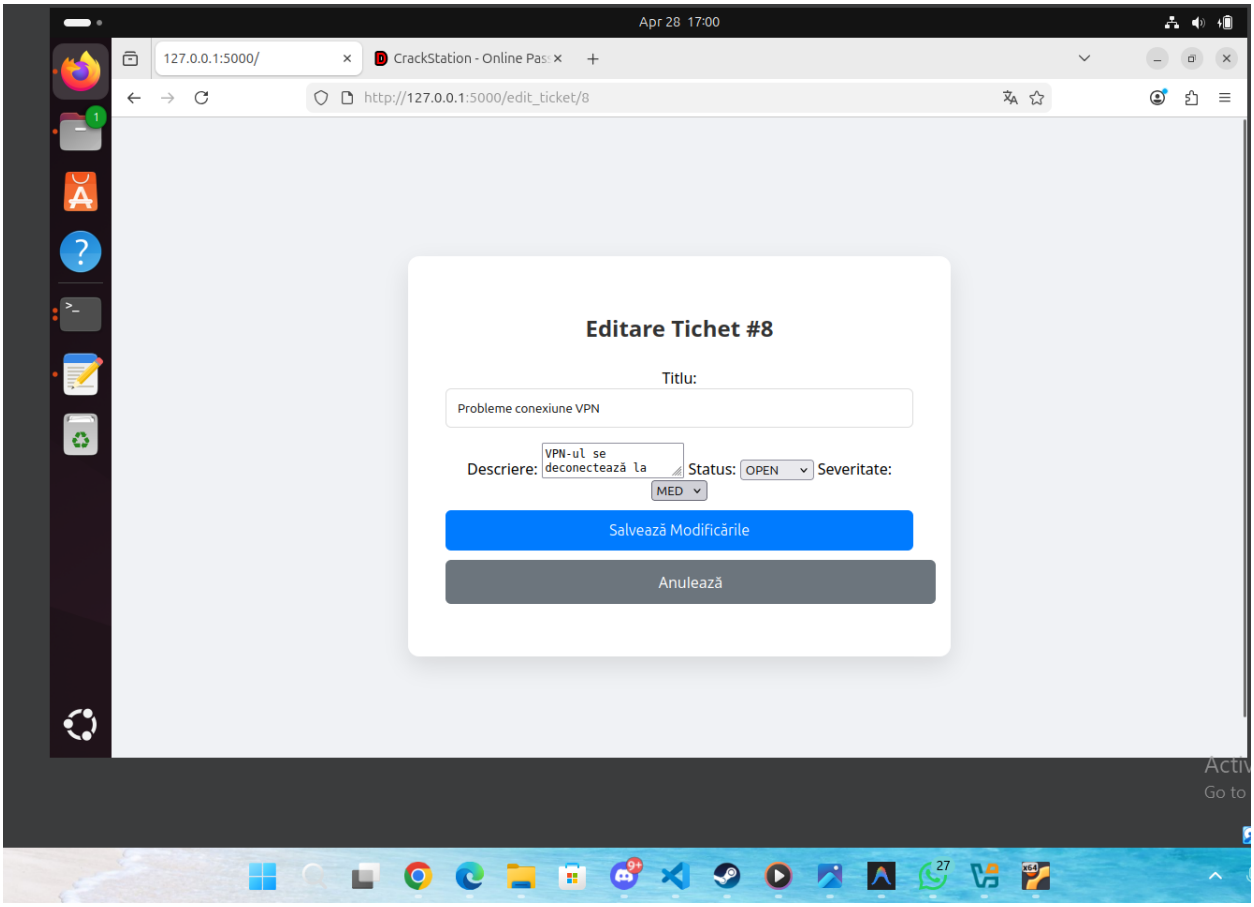


Fig. 16: test@test.com are acces la ticketele altor useri (1)

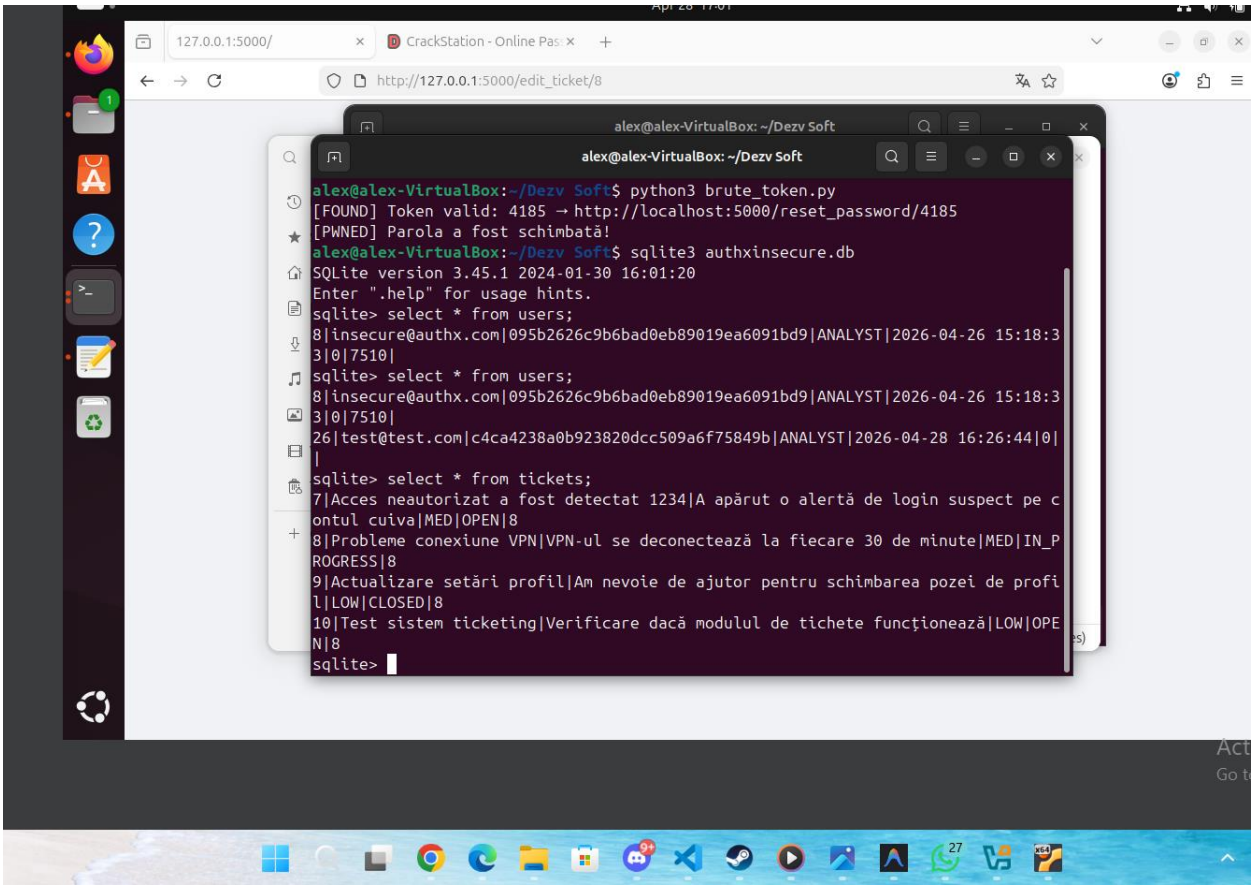


Fig. 17: test@test.com are acces la ticketele altor useri (2)

3. **Execuția Modificării:** Deoarece serverul din `app_insecure.py` verifică doar dacă ticketul există, nu și cine îl deține., așa că am reușit să modific ticketul atribuit unui alt user.

Editarea resurselor altor utilizatori duce la o escaladare de privilegii pe orizontală, iar atacul demonstrează o breșă în logica de control al accesului, permițând oricărui angajat să altereze date care nu îi aparțin.

7. Implementarea versiunii securizate (app_secure.py)

Fix-uri implementate:

Trecerea de la versiunea vulnerabilă la cea securizată a presupus o refactorizare completă a logicii de autentificare și gestionare a datelor. Scopul principal a fost neutralizarea vectorilor de atac demonstrați în faza de PoC prin implementarea unor controale de securitate recunoscute la nivel industrial.

7.1 Password Policy

Pentru a contracara atacurile bazate pe dicționare, am implementat o barieră de validare la nivel de server. Aceasta forțează utilizatorul să creeze parole cu entropie ridicată, făcând efortul de brute-force prohibitiv.

Am utilizat expresii regulate (Regex) pentru a verifica respectarea următoarelor criterii:

- Lungime minimă de 8 caractere;
- Prezența a cel puțin unei majuscule (A-Z);
- Prezența a cel puțin unei minuscule (a-z);
- Includerea a cel puțin unei cifre (0-9).

```
if request.method == 'POST':
        new_password = request.form.get('password')

        if len(new_password) < 8:
            error_message = "Parola trebuie să aibă minim 8 caractere!"

        elif (not re.search(r'[A-Z]', new_password) or
              not re.search(r'[a-z]', new_password) or
              not re.search(r'[0-9]', new_password)):
            error_message = "Parola trebuie să conțină litere mari, mici și
numere!"
```

7.2 bcrypt

Am înlocuit algoritmul MD5 cu **bcrypt**, un algoritm de hashing adaptiv care include automat un "salt" pentru fiecare parolă. Mai mult, am eliminat riscul de SQL Injection prin utilizarea interogărilor parametrizate.

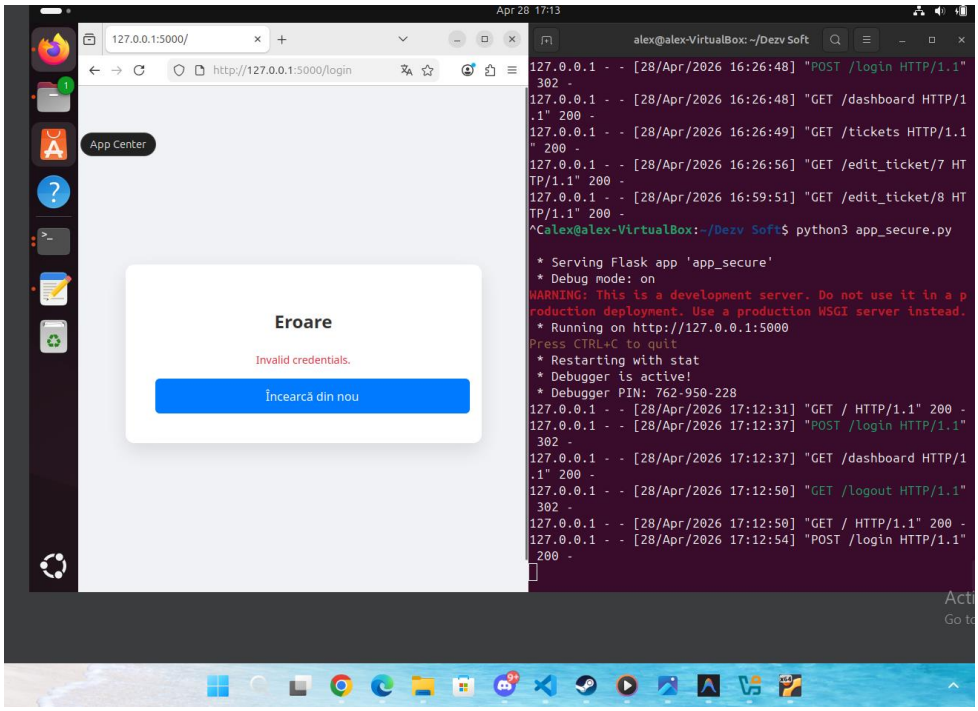
Bcrypt utilizează un factor de cost care determină numărul de iterații, făcând hash-ul imun la atacurile cu Rainbow Tables și extrem de rezistent la încercările de brute-force hardware (GPU/ASIC).

```
else:
            hashed = bcrypt.hashpw(new_password.encode('utf-8'),
bcrypt.gensalt()).decode('utf-8')
            db.execute("""
                UPDATE users
                SET password_hash = ?, reset_token = NULL, token_expiry = NULL
                WHERE id = ?
```

```
        """, (hashed, user['id'])) db.commit()
```

7.3 Generic Errors

Pentru a preveni enumerarea utilizatorilor, aplicația returnează acum un mesaj identic: **"Invalid credentials"**, indiferent dacă email-ul nu există în bază sau doar parola este greșită. Astfel, un atacator nu mai poate confirma prezența unui cont valid în sistem.



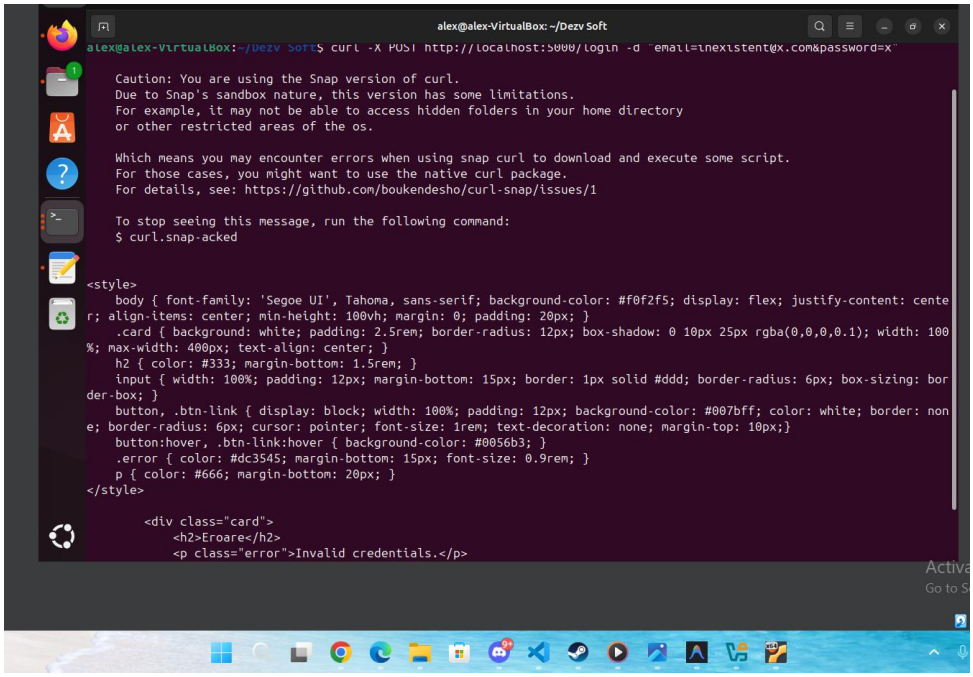


Fig. 18 & 19: Mesaj unic (1)

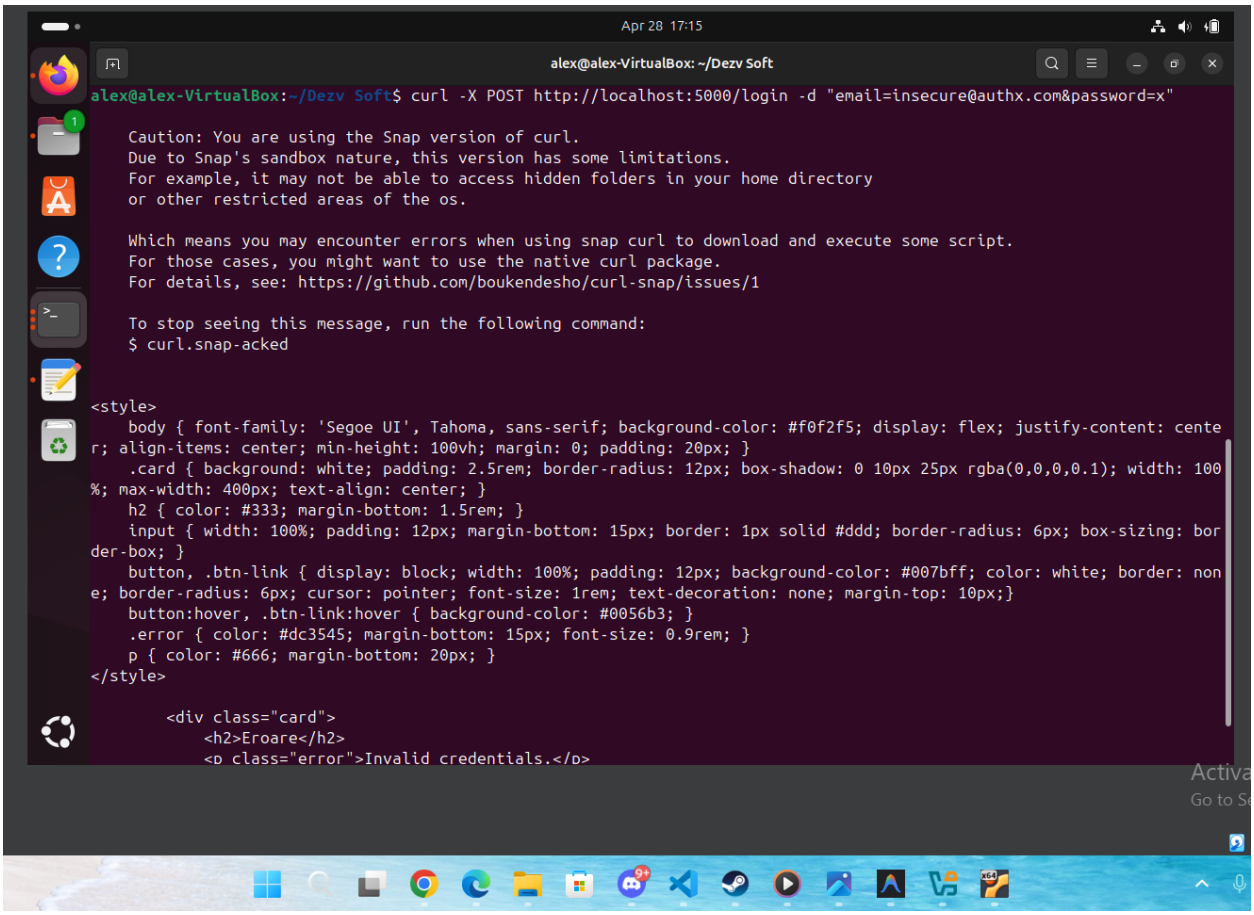


Fig. 20: Mesaj unic (2)

7.4 Dummy Hash

Într-un atac de tip "timing", un atacator măsoară timpul de răspuns al serverului. Dacă serverul răspunde mai repede când email-ul nu există (pentru că sare peste verificarea parolei), identitatea este expusă. Am implementat un "dummy hash", chiar dacă utilizatorul nu este găsit, serverul rulează o verificare de hash falsă pentru a menține timpul de răspuns constant.

```
DUMMY_HASH = bcrypt.hashpw(b"dummy_password", bcrypt.gensalt())

# câteva linii mai târziu ....

if user:
        valid = bcrypt.checkpw(password.encode('utf-8'),
user['password_hash'].encode('utf-8'))
    else:
        bcrypt.checkpw(password.encode('utf-8'), DUMMY_HASH)
        valid = False
```

7.5 Rate Limiting

Am implementat o politică de tip **5 încercări în 15 minute**.

- Dacă pragul este depășit, IP-ul și contul sunt blocate temporar.
- Acest mecanism face scriptul `brute_force.py` complet inutil, deoarece după primele 5 încercări, atacatorul este oprit forțat.

```
def is_rate_limited(ip):
    db = get_db()
    count = db.execute("""
        SELECT COUNT(*) FROM audit_logs
        WHERE ip_address = ?
        AND action = 'LOGIN_FAILED'
        AND timestamp > datetime('now', '-15 minutes')
    """, (ip,)).fetchone()[0]
    return count >= 5
```

7.6 Session Hardening

Am configurat cookie-urile de sesiune pentru a preveni furtul acestora (Session Hijacking):

- `HttpOnly=True`: Previne accesarea cookie-ului prin JavaScript (protecție contra XSS).
- `SameSite='Strict'`: Previne trimiterea cookie-ului în cereri cross-site (protecție contra CSRF).
- `Session Expiration`: Sesiunea expiră automat după o perioadă de inactivitate.
- `Regenerate ID`: La logare, ID-ul sesiunii este regenerat pentru a preveni Session Fixation.

```
app.permanent_session_lifetime = timedelta(minutes=15)

app.config.update(
    SESSION_COOKIE_HTTPONLY=True,
    SESSION_COOKIE_SECURE=True,
    SESSION_COOKIE_SAMESITE='Strict',
)
```

7.7 Secure Password Reset

Am abandonat PIN-ul de 4 cifre în favoarea unui token criptografic de mare entropie generat cu `secrets.token_urlsafe(32)`.

- **Impredictibilitate:** Spațiul de căutare al token-ului este de 2^{256} făcând atacul `brute_token.py` imposibil.

- **Expirare:** Token-ul este valabil doar 15 minute.

```
if user:
            token = secrets.token_urlsafe(32)
            expiry = datetime.now() + timedelta(minutes=15)
            db.execute("UPDATE users SET reset_token = ?, token_expiry = ? WHERE
id = ?", (token, expiry, user['id']))
            db.commit()
            send_reset_email(email, token)
```

7.8 Broken Access Control Fix

Am corectat logica de autorizare prin aplicarea controlului accesului la nivel de rând (Row-Level Security). La fiecare cerere de vizualizare sau editare, serverul verifică acum dacă `owner_id` al resursei corespunde cu `user_id` din sesiune. Un utilizator de tip `MANAGER` are permisiunea de a trece peste această verificare (override controlat), dar un `ANALYST` este strict limitat la propriile date.

```
if role == 'ANALYST':
        query += " AND owner_id = ?"
        params.append(user_id)
is_owner = (ticket['owner_id'] == session.get('user_id'))
    is_manager = (session.get('role') == 'MANAGER')
```

7.9 Audit Logging

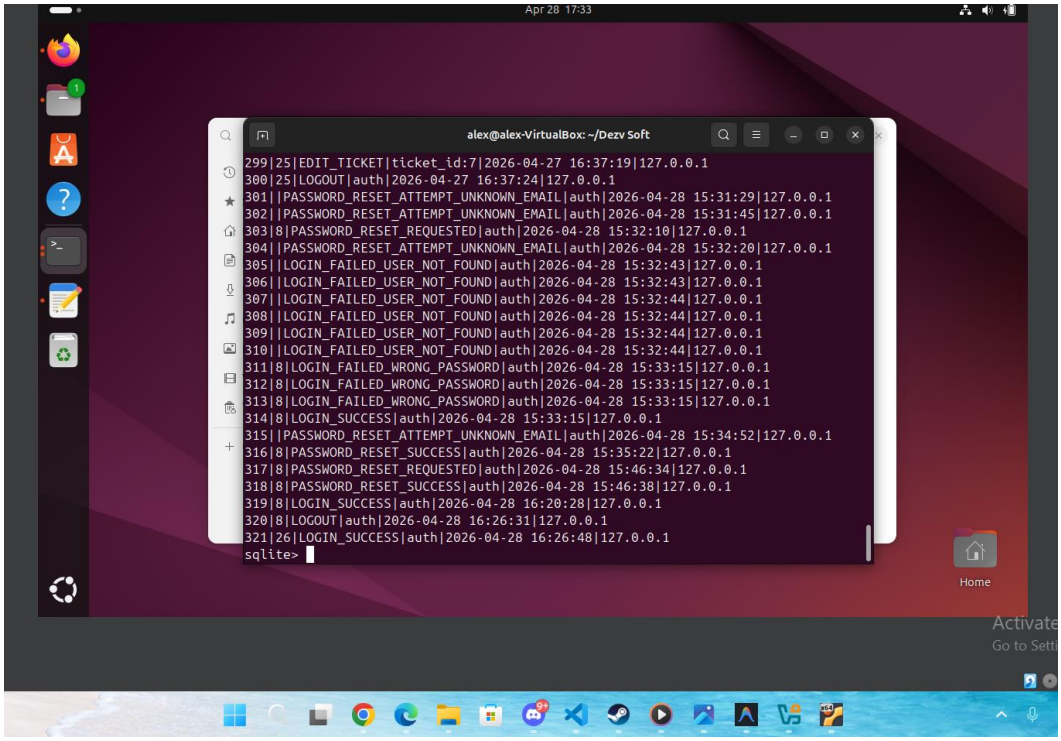


Fig. 21: audit logs

8. Re-testarea aplicației după fix-uri

Brute Force:

Blocare după 5 încercări.

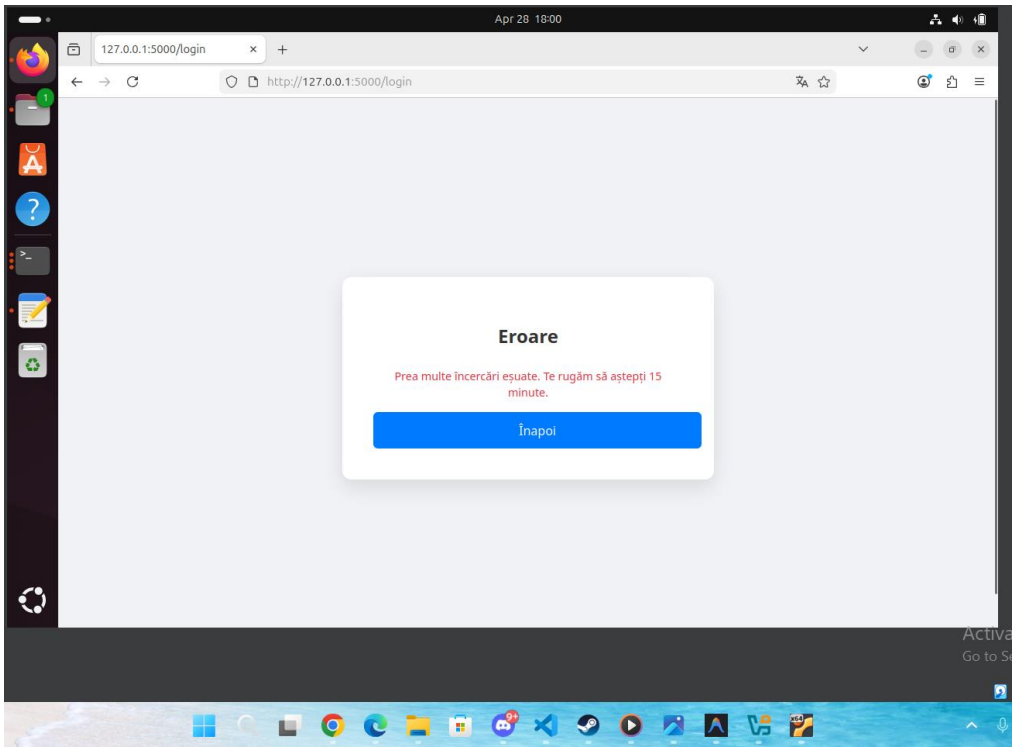
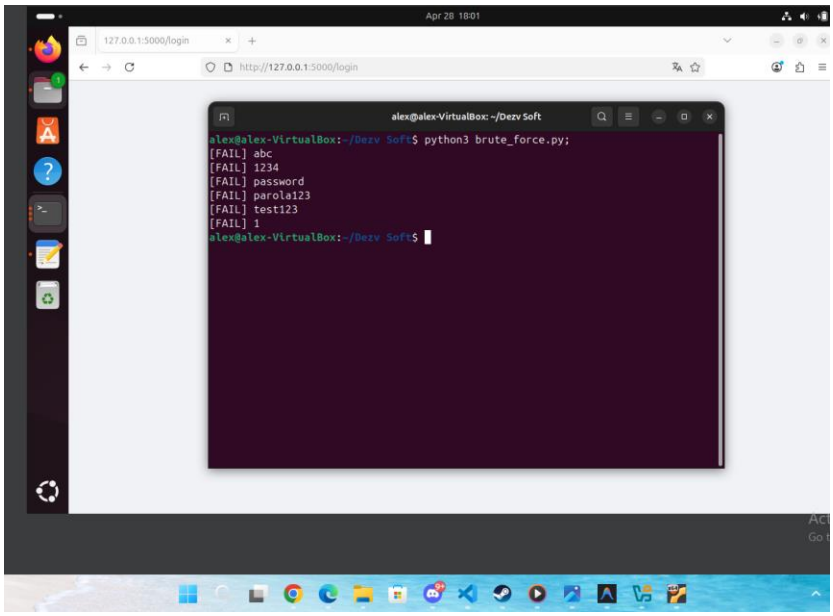


Fig. 22 & 23: Mesaj erroare pentru prea multe încercări eșuate de login



Token Attack:

Token imposibil de ghicit. Rularea programului nu afișează mesaj de succes:

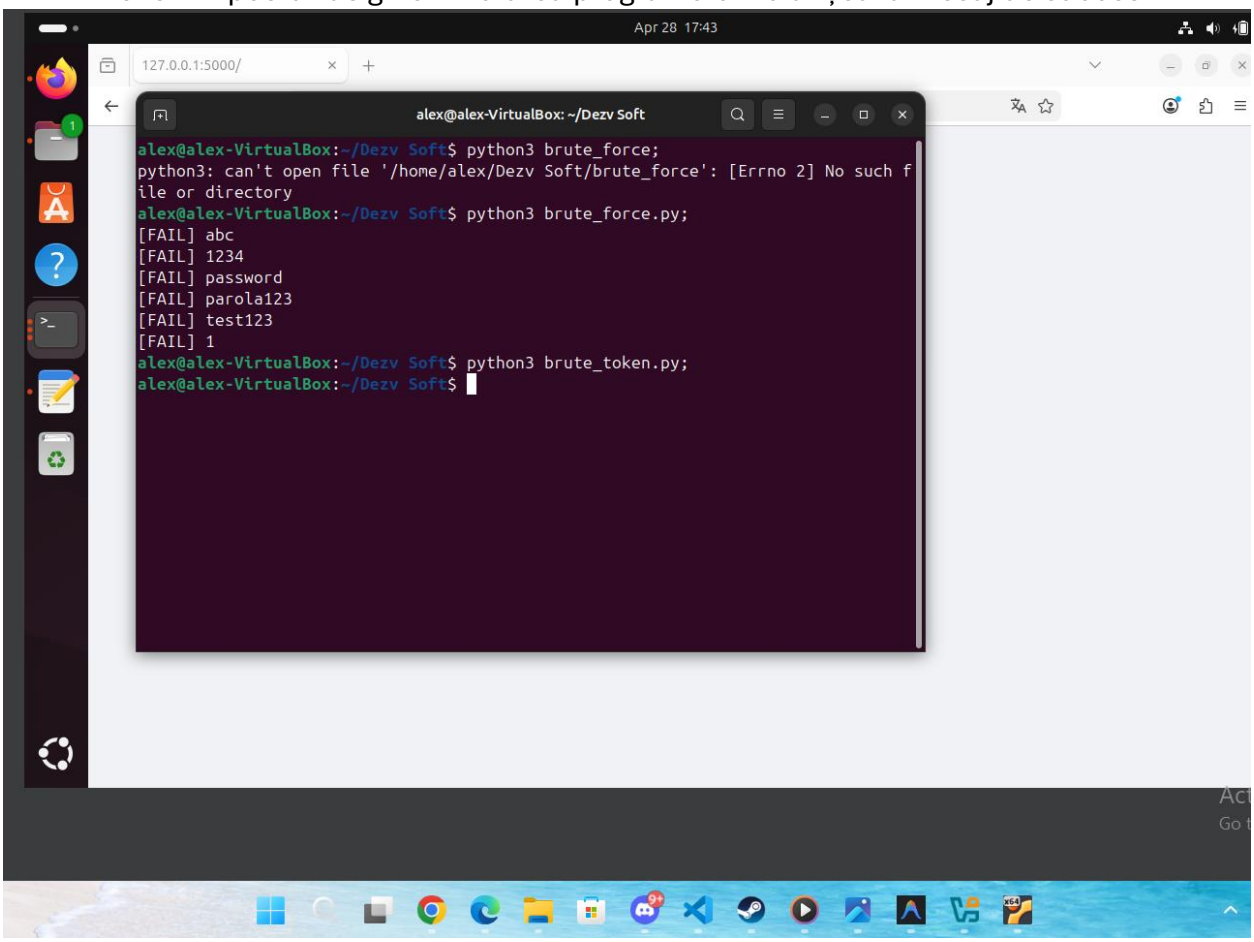


Fig. 24: Lispa mesajului de succes dovedește fail-ul atacului

IDOR:

```
is_owner = (ticket['owner_id'] == session.get('user_id'))
    is_manager = (session.get('role') == 'MANAGER')

    if not (is_owner or is_manager):
        return render_template_string(CSS_STYLE + '''
        <div class="card">
            <h2>Acces Interzis</h2>
            <p>Nu ai permisiunea de a edita acest tichet.</p>
            <a href="/tickets" class="btn-link">Înapoi la Tickete</a>
        </div>
        '''), 403
```

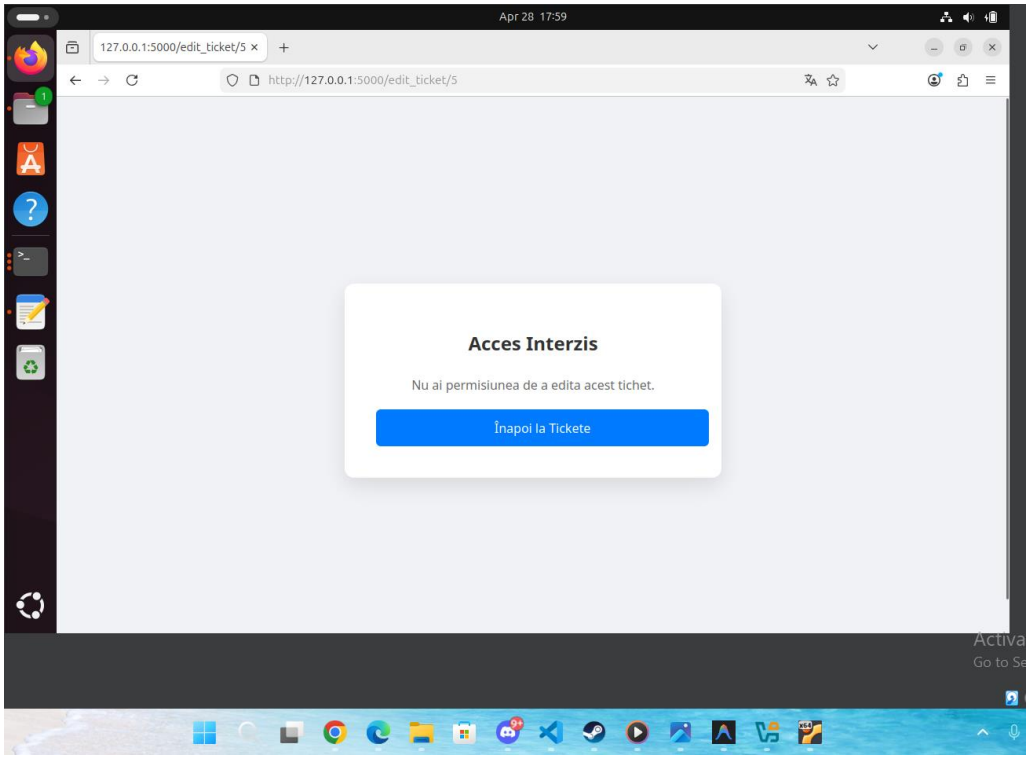


Fig. 25: Lispa mesajului de succes dovedește fail-ul atacului

User Enumeration:

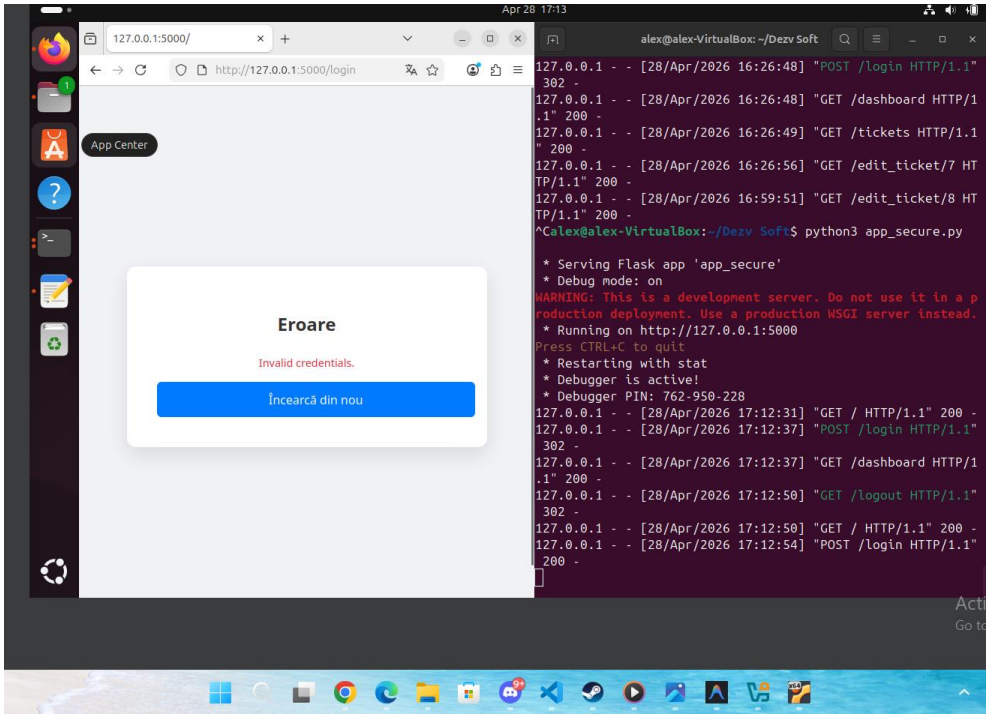


Fig. 26: Mesaj unic

9. Audit logging și trasabilitate

Exemple:

- LOGIN_SUCCESS
- LOGIN_FAILED
- PASSWORD_RESET_SUCCESS
- EDIT_TICKET_SUCCESS
- LOGOUT

Beneficii:

- forensic investigation
- brute force monitoring
- accountability

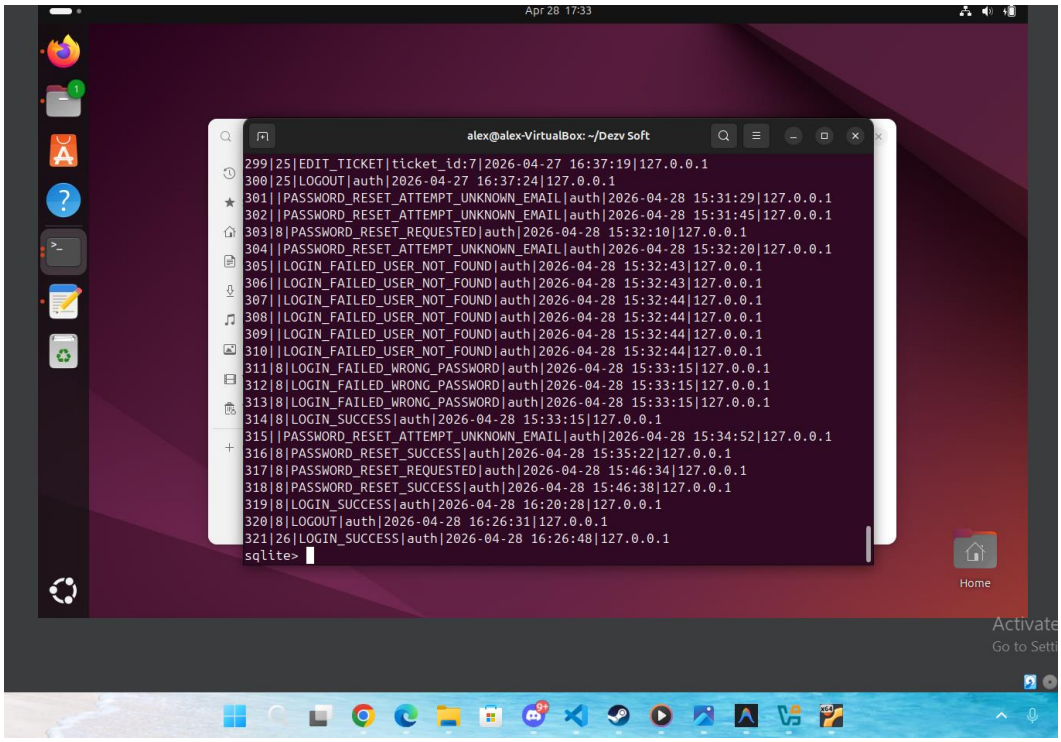


Fig. 27: Audit logs

---

10. Tabel comparativ – Insecure vs Secure

Categorie	Insecure	Secure
Password Storage	MD5	bcrypt
Password Policy	slabă	complexă

Categorie	Insecure	Secure
Brute Force	nelimitat	rate limit
User Enumeration	da	nu
Session Security	slabă	hardened
Reset Token	predictibil	random
Access Control	broken	enforced
Audit	limitat	complet

---

11. Concluzii

Am demostrat practic prin acest proiect cât de periculos poate deveni un mecanism de autentificare atunci când principiile de securitate sunt ignorate.

Lecțiile pe care le-am învățat:

- autentificarea trebuie tratată ca suprafață majoră de atac
- hashing-ul modern este obligatoriu
- rate limiting-ul este esențial
- mesajele generice reduc reconnaissance
- token-urile trebuie să fie random și expirabile
- controlul accesului trebuie aplicat server-side
- audit logging-ul este vital

Bibliografie

[1] OWASP Top 10
[2] Flask Documentation
[3] SQLite Documentation
[4] bcrypt Documentation
[5] Materiale curs DASS